

Understanding Requirement Engineering

Anil Mokhade 6 to 17 March 2026

Today we start a very important topic in Software Engineering — Requirements Engineering. Traditionally, requirements were understood as something humans think about, document carefully, and then pass to developers who implement the system.

But today we live in a very different technological world.

Between the moment a requirement is conceived and the moment the software is deployed, there now exists a new participant — Artificial Intelligence.

AI can now read requirements, refine them, generate designs, produce code, test systems, and even monitor software behavior in production.

Therefore, the classical boundary between ‘thinking about requirements’ and ‘building software’ is dissolving.

*Requirements Engineering is no longer just about writing documents. It is about designing **understanding itself** — between humans, machines, organizations, and users.*

*The classical view of requirements as a **human-only cognitive activity before coding** is no longer entirely valid.*

So when we study requirements today, we are not merely learning a phase in a waterfall model. We are studying the **intelligence layer of software development**.

What is changing?

1. requirement discovery
2. interpretation
3. validation, and
4. evolution.

Breaking the Classical Thought:

In Software Engineering, the success of any project is not just about writing code-it's about building the right product. And to build the right product, we must first understand the right requirements.

Traditionally, requirement engineering was about fixed documents and rigid processes. Then came Agile, revolutionizing how we define and adapt requirements through iterative feedback. But today, we are witnessing yet another transformation - AI is no longer just a tool; it is becoming a collaborator, reshaping how we gather, refine, and even generate requirements dynamically. So as we move beyond Agile and Scrum, let us explore how Requirements Engineering is evolving in this age of intelligence-driven development.

The AI Disruption in Requirements Engineering

Today AI is entering **three important places in the lifecycle**.

1. Before Requirements (Problem Understanding)

AI can analyze:

- customer feedback
- logs
- user behavior
- social media
- support tickets

Thus requirements are **discovered automatically**.

Example:

Amazon identifies features from:

- clickstream data
- search patterns
- abandoned carts

This is **data-driven requirements discovery**.

2. During Requirements Specification

AI tools can now:

- summarize stakeholder interviews
- generate user stories
- detect inconsistencies
- suggest missing requirements

Example tools:

- ChatGPT
- Claude
- GitHub Copilot
- Notion AI

Example task:

From a paragraph description, AI generates:

User Story:

As a campus student,
I want to generate a digital gate pass,
So that security can verify my exit.

3. Between Requirements and Implementation

This is the **most revolutionary part**.

AI can now convert requirements into:

- architecture suggestions
- UML diagrams
- code
- test cases

Example:

Requirement → Prompt → Working prototype.

Thus the gap between **idea and implementation collapses**.

The New Reality

So the pipeline is evolving from:

Classical Pipeline

Human Requirement → Design → Code
to

AI-Augmented Pipeline

Human Intent

↓

AI-assisted Requirement Interpretation

↓

AI-assisted Design

↓

AI-assisted Code Generation

↓

Human + AI Verification

This creates a new discipline:

AI-Augmented Requirements Engineering

Conceptual Shift

three deep changes:

1. Requirements are becoming dynamic

Earlier:

Requirements → fixed specification

Now:

Requirements → continuously evolving understanding
because software learns from:

- user feedback
- runtime behavior
- AI analytics

2. Requirements are increasingly probabilistic

Traditional software:

IF condition → DO action

AI systems:

Given probability distribution → choose optimal action

Example:

Recommendation systems.

Requirements are now shared between humans and machines

Before:

Human → defines requirement

Machine → executes

Now:

Human + AI → co-create requirements

Machine → partly designs and executes

The Hospital Software Story

Imagine a hospital wants to build a new digital system.

Twenty years ago, the process was very clear.

Doctors and administrators would sit with software engineers.

They would say things like:

We need patient registration.

We need billing.

We need laboratory reports.

Engineers would carefully write these requirements into a document called SRS — Software Requirements Specification.

Then the developers would go away for six months and build the system.

But now imagine the same hospital today.

The hospital has thousands of patient records, diagnostic reports, appointment logs, insurance claims, and medical imaging data.

Instead of asking doctors only, an AI system analyzes millions of hospital interactions.

It discovers patterns like:

- Patients cancel appointments after long waiting times.
- Billing errors occur frequently in emergency admissions.
- Lab reports are often delayed during peak hours.

Suddenly the requirements are not just what people say they need — but what **data reveals they actually need.**

The AI might even suggest:

‘You should build a predictive waiting-time system.’

Or

‘You should implement automated insurance verification.’

Now tell me — who discovered the requirement?

The doctor?

The administrator?

Or the AI?

This is the new world of Requirements Engineering.”

“Therefore, when we study Requirements Engineering in this course, we will study two worlds simultaneously:

1.The classical discipline of Requirements Engineering

2.The emerging AI-augmented requirements ecosystem

Our goal is not just to write requirement documents — but to understand **how software systems understand problems.**”

Connect to with our CSO

Framework

We can beautifully connect your CSO model here.

Concept

What is the problem?

What is the user intent?

Operation

How does the system run in the real world?

- deployment
- feedback
- monitoring
- continuous requirement evolution

Thus:

Requirement Engineering = Concept Layer
of Software Systems

Strategy

How should the system solve it?

- architecture
- data
- algorithms
- AI models

In the early days of computing, programming was the hardest part.

Today coding is becoming easier because AI can help us write code.

But understanding the **right problem to solve** — that remains the most difficult part.

That is why Requirements Engineering will remain the *intellectual heart of Software Engineering*.

If AI can generate code from requirements, what becomes the most valuable skill for a software engineer?

.....

Classical Software Engineering Pipeline

Problem Understanding
(Stakeholders)
↓
Requirements Engineering
(Elicitation, Analysis, SRS)
↓
System Design
↓
Implementation
↓
Testing
↓
Deployment
↓
Maintenance

Characteristics:

- Requirements written by humans
- Documents are primary artifact
- Long development cycles
- Changes expensive

AI-Augmented Software Engineering Pipeline

Human Intent / Problem Context
↓
AI-assisted Requirement
Discovery
(Data, Logs, Feedback Analysis)
↓
AI-assisted Requirement
Structuring
(User Stories, UML, Domain
Models)
↓
AI-assisted Design & Code
Generation
↓
Continuous Testing & Monitoring
↓
Runtime Feedback (AI Analytics)
↓
Requirement Evolution

Characteristics:

- Requirements emerge from **human + data + AI**
- Faster prototyping
- Continuous requirement evolution
- AI assists design and coding

Key Insight (important)

Traditional Thinking

Requirement → Design → Code

New Thinking

Intent → Understanding → Exploration → Implementation

Where AI participates in **understanding and exploration.**

Let us take problem to solve:

“Students often face difficulty in getting gate passes from the campus security office when they need to leave the campus. The process is manual, involves paperwork, and causes delays during peak hours.”

What are the requirements here?

Prompt:

Extract functional and non-functional requirements from the following description.

The AI output.

Example:

Functional requirements:

- Students should request gate pass digitally
- Security should verify student identity
- System should record exit time
- Approval workflow should exist

Non-functional requirements:

- Authentication required
- Response time < 3 seconds
- System must be accessible via mobile

Ask AI:

Convert these requirements into user stories.

Example output:

As a student,
I want to request a digital gate pass,
So that I can leave the campus without waiting in queues.

This is requirement transformation.

What are the dangers of relying completely on AI here?

Are these the possible answers?

- misunderstanding domain context
- missing constraints
- privacy issues
- security concerns

So, can we say?:

AI assists requirements engineering, but **domain knowledge remains essential.**

“The biggest failure in software projects rarely happens during coding.
It happens when the **wrong problem is understood**.
A perfectly implemented wrong requirement still produces a useless system.”

If AI can write code, what becomes the most important skill for software engineers?

Problem understanding and requirement thinking.

Understanding the real problem:

When early software engineering methods like Waterfall were designed, engineers believed that requirements could be understood clearly before development begins.
But in reality, something interesting kept happening in software projects:
requirements kept changing.”

Why do requirements change?

May be because:

- users change their mind
- technology evolves
- business conditions change
- developers misunderstand

“Because requirements change frequently, a new philosophy of software development emerged.
This philosophy is called **Agile Software Development**.”

Elementary Definition of Agile

Let us take simple definition first.

Agile Software Development

Agile is a software development approach in which **requirements and solutions evolve through continuous collaboration between developers and stakeholders**, and software is built **incrementally in short development cycles called iterations or sprints**.

Agile = building software in small steps while continuously refining requirements.

The Core Idea of Agile

In classical development:

Understand requirements completely → build system

But Agile assumes:

Understand part of the requirement → build → learn → refine requirement

Thus development becomes a **learning process**.

Expressing this visually:

Classical Thinking

Requirements → Design → Code → Test

Agile Thinking

Understand → Build → Learn → Refine → Repeat

This is **continuous requirement discovery**.

Agile and Requirements Engineering

Connecting Agile with requirements.

In Agile:

Requirements are not written as long documents initially.

Instead they appear as:

User Stories

Example:

As a patient,
I want to book a hospital appointment online,
So that I do not need to wait in queue.

User stories are:

- short
- user-focused
- evolving

Thus in Agile:

Requirements are continuously discovered and refined.

Why Agile Emerged (Historical Insight)

Agile emerged because of three realities:

1. Requirements uncertainty

Users do not fully know what they want.

2. Technology uncertainty

New technologies appear during development.

3. Market uncertainty

Business goals change.

Therefore:

Agile treats **requirements as evolving knowledge** rather than fixed truth.

Where AI Fits into Agile Today

Agile already assumes **continuous requirement refinement**.

AI makes this even faster.

AI can help in:

Requirement Discovery

AI analyzes:

- user feedback
- product usage data
- logs
- customer complaints

Example:

Netflix discovering new features from user behaviour.

User Story Generation

AI tools can generate:

- user stories
- acceptance criteria
- test cases

Example prompt:

Convert these requirements
into Agile user stories.

Continuous Feedback

AI analyzes runtime behavior.

Example:

- system logs
- error patterns
- user interactions

These insights generate **new requirements automatically**.

Rapid Prototyping

AI can generate:

- UI prototypes
- code
- architecture suggestions

Thus Agile cycles become
much faster.

Agile + AI = Continuous Requirement Intelligence

Earlier:

Human → defines requirement

Now:

Human + Data + AI → discover requirement

Thus requirements become **data-driven**.

“Agile made software development adaptive.

AI now makes it **intelligent and adaptive**.”

The Food Delivery App Story

Imagine a startup building a food delivery application.

Initially they define requirements:

- restaurant listing
- order placement
- payment gateway

They release the system.

After deployment they notice something interesting in user behavior.

Many users repeatedly order the same food items.

Data analysis suggests a new feature:

“Quick Reorder Button.”

This feature was not in the original requirement.

It emerged from **user behavior data**.

In the past this insight might take months.

Today AI analytics can detect such patterns automatically.

Thus the requirement emerges dynamically.

Who discovered the requirement?

Users?

Developers?

AI?

Insight

“Traditional software engineering assumed requirements are known before development.

Agile assumes requirements are discovered during development.

AI now allows requirements to be discovered **continuously from real-world data**.”

How to Connect This to Your CSO Framework

CSO Layer	Role
Concept	Understanding the problem and requirements
Strategy	Choosing Agile, architecture, and development approach
Operation	Iterative deployment and feedback

Thus:

Agile operates across Concept–Strategy–Operation continuously.

“So before we go deeper into requirements engineering techniques like elicitation, analysis, and specification, we must remember one important fact:

Requirements are not just written — they are discovered.”

“If AI can understand requirements, why do we need to learn Requirements Engineering?”

We just saw that Agile assumes requirements are discovered gradually.
But even Agile does not eliminate the need for clear understanding of requirements.
In fact, Agile requires **even more careful thinking about requirements**, because development happens in rapid iterations.

So the key question remains:

“If software development methodologies have evolved so much — Waterfall, Spiral, Agile, DevOps — why do software projects still fail?”

“Let us look at the evidence.”

Software Project Failures

Introduce the **Standish Group CHAOS Report** (a famous reference).

Studies of thousands of software projects show that the most common reasons for failure include:

Reason	Approximate Impact
Incomplete requirements	very high
Changing requirements	very high
Lack of user involvement	high
Unrealistic expectations	high

“A very large number of software failures originate in **requirement misunderstanding**.”

The Fundamental Lesson

We can articulate this:

“Technology failures are rarely technological.

They are usually **understanding failures**.”

Requirement Engineering

Requirement Engineering is the systematic process of **discovering, analyzing, documenting, validating, and managing the needs and constraints of stakeholders for a software system**.

Reference:

Sommerville – Software Engineering

How Is This Connected With AI

“Today AI tools can help generate requirements, summarize stakeholder discussions, and even propose system designs.

But AI does not automatically understand the domain context, stakeholder conflicts, or real-world constraints.”

Examples:

AI may miss:

- regulatory constraints
- organizational politics
- safety considerations
- ethical issues

Thus:

“AI can assist requirements engineering — but it cannot replace **human judgment about the problem space**.”

What Insight Can We Draw from this?

“Coding errors break software modules.
Requirement errors break entire systems.”

Conclusion:

“The difficulty in software engineering is not writing code.

The real difficulty is **understanding what problem the software must solve.**”

This insight is very important.

Fred Brooks (*The Mythical Man-Month*) emphasized this strongly.

Reference quote:

“The hardest single part of building a software system is deciding precisely what to build.”

“So before we study requirement elicitation techniques or requirement documentation formats, we must first understand why requirement clarity is so critical.

Because in software engineering, **a wrong requirement implemented perfectly is still a failure.**”

“Before we study the actual activities of Requirements Engineering, we understood why this discipline became necessary, why Agile changed its treatment, and why AI does not remove the need for it.”

Introduction to Requirements Engineering (RE)

- **Definition:** The process of defining, documenting, and maintaining software requirements.
- **Importance:** Ensures software meets user needs and business goals.
- **Key Activities:** Elicitation, Analysis, Specification, Validation, and Management.
- **Challenges:** Ambiguous requirements, evolving needs, stakeholder conflicts.

Classical View of Requirements Engineering

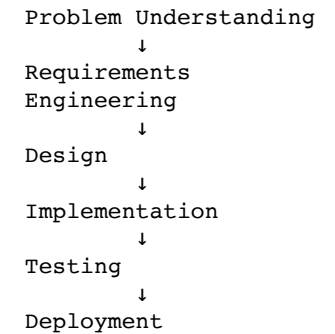
• Waterfall Model and Requirements Engineering:

- Requirements fully specified upfront.
- **Types of Requirements:**
 - **Functional:** What the system should do (e.g., login functionality).
 - **Non-functional:** Performance, security, usability constraints.
 - **Domain Requirements:** Specific to the problem domain.
- **Requirement Documents:** Software Requirements Specification (SRS).
- **Validation Techniques:** Prototyping, formal verification.

• Limitations of Traditional Requirements Engineering:

- Late discovery of issues.
- Inflexibility to changes.
- Communication gap between stakeholders and developers.

Traditionally the software lifecycle looked like:



Requirements were assumed to be:

- **Human generated**
- **Explicit**
- **Relatively stable**
- **Captured in documents (SRS)**

Typical activities:

1. Requirement Elicitation
2. Requirement Analysis
3. Requirement Specification
4. Requirement Validation
5. Requirement Management

Requirement Elicitation:

This is the first place where AI is already changing practice, but it is also the place where **human judgment remains most critical**.

There are three layers:

1. **Classical elicitation (human-centered)**
2. **AI-assisted elicitation (data + AI tools)**
3. **Human responsibility in AI-supported elicitation**

- **AI changes the mechanics of elicitation but not the intellectual responsibility.**

Requirement Elicitation Defined:

Requirement elicitation is the process of **discovering and understanding stakeholder needs, goals, constraints, and expectations for a system.**

Key point to emphasize:

Requirements are rarely handed over clearly. They must be **elicited**.

What is the meaning of the word *elicitation*?

Elicit means:

- to draw out
- to uncover
- to bring hidden knowledge to the surface

This is important because stakeholders often **do not articulate requirements explicitly**.

It is called **requirement elicitation** rather than **requirement discovery** because requirements are usually **not simply “found” in the world like objects**; they must be **drawn out from stakeholders through interaction, interpretation, and negotiation**.

Meaning of the Word Elicit

The word **elicit** comes from the Latin *elicere*, meaning:

“to draw out” or “to bring forth.”

In common English usage, it means:

- to obtain information by asking questions
- to bring out something hidden or implicit

Examples outside software:

- A teacher **elicits answers** from students.
- A therapist **elicits feelings** from a patient.

In both cases, the information already exists **in the person's mind**, but it must be **brought out carefully**.

That is exactly what happens in Requirements Engineering.

Why Discovery Is Not Fully Accurate

The word **discovery** suggests something like:

- discovering a new planet
- discovering a mineral deposit
- discovering an existing fact

Discovery implies the thing **already exists objectively**, and you simply find it.

But requirements are rarely like that.

They are usually:

- incomplete
- partially understood
- sometimes conflicting
- sometimes evolving

So requirements are **not waiting to be discovered like fossils**.

They are **constructed through interaction**.

Requirements Often Do Not Exist Fully Formed

A critical insight is this:

Stakeholders themselves often do not fully know their requirements.

Fred Brooks made this point clearly:

Clients **do not know the questions that must be answered** until the system begins to take shape.

Examples:

A hospital administrator may say:

“We need a hospital management system.”

But after discussion we learn they actually need:

- appointment scheduling
- electronic medical records
- billing integration
- insurance claim processing
- laboratory report management

The real requirements emerge **through dialogue**.

Thus they must be **elicited**, not merely discovered.

Requirements Are Socially Constructed

Another important reason is that requirements involve **multiple stakeholders**.

Each stakeholder may have a different view.

Example: Hospital system

Doctor wants:

- quick access to patient history

Administrator wants:

- billing accuracy

IT department wants:

- system security

Patients want:

- shorter waiting time

These requirements must be **negotiated and reconciled**.

So the final requirement set is **constructed**, not discovered.

Requirement Elicitation Is a Knowledge Process

Requirement elicitation involves several activities:

- asking questions
- observing work practices
- analyzing existing systems
- identifying hidden constraints
- clarifying ambiguous statements
- negotiating conflicts

These activities are closer to **knowledge extraction** than discovery.

In fact, in Knowledge Engineering and AI literature the term **knowledge elicitation** is also used.

This is not a coincidence.

Requirements Engineering borrowed the same concept.

Where “Discovery” Actually Fits

However, the term **requirement discovery** is not completely wrong.

It is often used in modern contexts such as:

- product discovery
- design thinking
- user research
- data-driven requirement discovery

Especially today, with AI and analytics, systems may discover patterns such as:

- frequent user complaints
- performance bottlenecks
- user behavior anomalies

These may suggest **new requirements**.

So discovery is increasingly part of the process.

But classical RE emphasized **elicitation** because interaction with stakeholders is central.

“Requirements are rarely sitting somewhere waiting to be discovered. They exist partly in people’s minds, partly in organizational processes, and partly in system usage patterns.
Therefore we must draw them out carefully.
That is why we call it **requirement elicitation**.”

A Helpful Analogy

You can give a simple analogy.

Doctor–Patient Interaction

A patient visits a doctor and says:

“I feel unwell.”

The doctor must ask questions:

- Where exactly does it hurt?
- When did it start?
- What triggers the pain?
- What makes it worse?

The doctor is **eliciting symptoms**.

The illness is not discovered instantly; it emerges through questioning.

Requirement elicitation works the same way.

A Modern Perspective (AI Era)

Today we might say:

Requirements come from three sources:

Source	Process
Stakeholders	Elicitation
System usage data	Discovery
AI analysis	Insight generation

So modern RE includes both:

elicitation + discovery.

But historically and conceptually, **elicitation captures the human interaction aspect better**.

“Software requirements are not simply discovered.
They are carefully elicited, interpreted, negotiated, and refined.”

How requirements become engineering artifacts that can guide design, testing, and deployment — even in the AI era?

Requirement Maturation Pipeline

Before discussing individual topics, we will see how a requirement evolves.

Example pipeline:

Raw Need
↓
Elicited Requirement
↓
Analyzed Requirement
↓
Specified Requirement
↓
Validated Requirement
↓
Managed Requirement (through lifecycle)

In modern software development, including AI-based systems, requirements move through this **maturation pipeline**.

This will make the rest of the topic much clearer.

Requirement Analysis

- **elicitation** gives raw material.

Analysis transforms it into **structured understanding**.

What happens in Requirement Analysis

- clarify ambiguity
- detect conflicts
- identify dependencies
- prioritize requirements
- check feasibility
- classify requirements

Example transformation:

Raw statement:

“The system should respond quickly.”

Analysis reveals:

- response time < 2 seconds
- peak load 1000 users
- mobile network constraints

This stage converts **vague statements** → **engineering statements**.

In the AI World

AI can assist requirement analysis by:

- detecting duplicate requirements
- identifying inconsistencies
- clustering similar requirements
- suggesting requirement categories

Example:

AI tools can analyze requirement documents and detect:

- conflicting statements
- missing dependencies
- unclear terms

But humans must still interpret **context and trade-offs**.

Requirement Specification

Analysis clarifies requirements.

Specification **documents them precisely**.

Specification answers:

- What exactly must the system do?
- Under what conditions?
- With what constraints?

Traditional formats:

- Software Requirement Specification (SRS)
- User Stories
- Use Cases
- Domain models
- Interface specifications

AI's Role in Specification

AI tools can now:

- convert informal descriptions into user stories
- generate use cases
- suggest acceptance criteria
- draft requirement documents

Example prompt:

Convert the following requirement into a user story and acceptance criteria.

AI output:

User Story
Acceptance Criteria
Edge cases

However:

AI may miss:

- domain constraints
 - regulatory requirements
 - safety considerations
- So human oversight remains essential.

Requirement Validation

Specification does not guarantee correctness.

Validation asks:

Are these requirements the right ones?

Validation checks:

- correctness
- completeness
- consistency
- feasibility
- testability

Example questions:

- Can this requirement be tested?
- Is it ambiguous?
- Does it conflict with another requirement?

AI and Requirement Validation

AI can assist validation through:

- consistency checking
- automated test case generation
- simulation
- requirement traceability

Example:

AI can analyze requirement statements and detect:

- vague terms like “fast”, “user-friendly”
- missing constraints
- duplicate requirements

But validation often requires **stakeholder judgment**.

Requirement Management

This is often the most underestimated activity.

Requirements **change continuously**.

Sources of change:

- user feedback
- regulatory changes
- technological changes
- business strategy
- system usage data

Requirement management includes:

- version control
- traceability
- change impact analysis
- prioritization

AI and Requirement Management

AI can assist in:

- tracking requirement dependencies
- predicting impact of requirement changes
- analyzing usage data to suggest new requirements
- detecting requirement drift

Example:

AI analyzing logs may reveal:

- users rarely use certain features
- unexpected user behavior patterns

These insights trigger **new requirements**.

Concept of Continuous Requirements Engineering

- important modern idea.

Traditional view:

Requirements → Design → Implementation

Modern view:

Requirements ↔ Development ↔ Deployment ↔ Feedback

Requirements evolve continuously.

This is sometimes called:

Continuous Requirements Engineering

Full Modern RE Loop

Stakeholder Needs



Requirement Elicitation



Requirement Analysis



Requirement Specification



Requirement Validation



Development & Deployment



User Feedback + Data



Requirement Evolution

AI contributes at multiple points.

Practical Exercise

To internalize this process, let us do a small activity.

Example system:

Campus Hospital Management System

Perform:

Step 1 — Analysis

Clarify vague requirement:

“The system should support patient registration.”

You must define:

- data fields
- workflow
- constraints

Step 2 — Specification

Write a user story.

Step 3 — Validation

Identify potential ambiguity.

Step 4 — Management

Discuss possible future changes.

Example System: Campus Hospital Management System

Initial vague requirement:

“The system should support patient registration.”

This statement is **too vague to guide system design or testing**.

So we move through the Requirement Engineering pipeline.

Step 1 — Requirement Analysis

Goal: Convert the vague statement into **clear system understanding**.

1. Identify Required Data Fields

What information must be captured.

Possible answer could be:

Patient Identification

- Patient ID (auto-generated)
- Full name
- Date of birth
- Gender
- Contact number
- Email ID

Institutional Information

- Student ID / Employee ID
- Department
- Hostel / residence information

Medical Information

- Blood group
- Known allergies
- Chronic conditions
- Emergency contact

Administrative Information

- Registration date
- Registration type (new / returning)
- Insurance details (if applicable)

These fields are **derived during analysis**, not directly stated in the requirement.

Define the Workflow

What sequence of steps happens during registration?

Possible workflow:

Patient arrives at hospital

↓

Identity verification

↓

Check if patient already exists

↓

If new patient → create patient record

↓

Capture required information

↓

Generate patient ID

↓

Store record in database

↓

Registration confirmation

This workflow reveals **system behavior**.

Identify Constraints

What constraints might apply?

Examples:

Security Constraints

- Only authorized staff can register patients

Privacy Constraints

- Patient data must comply with privacy regulations

Performance Constraints

- Registration should complete within 2 minutes

Data Integrity Constraints

- Duplicate patient records must be avoided

Constraints often emerge only during **analysis**.

Requirement Specification

Now convert the analyzed requirement into **formal specification**.

One simple approach is a **User Story**.

User Story

As a hospital receptionist,

I want to register new patients and create their medical records,

So that doctors can access patient information during consultation.

Acceptance Criteria

You should also see acceptance criteria.

Example:

1. System shall generate a unique patient ID for each registered patient.
2. System shall store patient demographic and medical information.
3. System shall prevent duplicate registration of the same patient.
4. System shall allow authorized staff to edit patient information.

Specification ensures the requirement is **clear and testable**.

Requirement Validation

Is the requirement **correct and complete**?

Look for **ambiguities**.

Possible ambiguities:

Ambiguity 1

What defines a **patient**?

- student?
- faculty?
- staff?
- external visitor?

Ambiguity 2

Who is authorized to register patients?

- receptionist
- nurse
- doctor

Ambiguity 3

What happens if a patient already exists?

- update record?
- merge records?
- reject registration?

Ambiguity 5

Offline scenario

What if the system is unavailable?

- manual registration?
- later synchronization?

Ambiguity 4

What data fields are mandatory?

Example:

- phone number required?
- blood group required?

Validation ensures requirements are **clear before development begins**.

Requirement Management

- **requirements evolve over time**.

What future changes might occur?

Possible changes:

Change 1 — Online Registration

Students may register before arriving at the hospital.

Change 2 — Integration With Institute Systems

Patient details automatically retrieved from:

- student database
- employee database

Change 3 — Emergency Registration

Quick registration for emergency cases.

Change 4 — Mobile App Integration

Patients register using mobile app.

Change 5 — AI-Based Triage

System may later capture symptoms and suggest urgency level.

Requirement Management ensures:

- traceability
- change tracking
- version control

Summarization of the process

Vague Need



Requirement Analysis



Structured Requirement



Specification



Validation



Requirement Management

“A single sentence requirement often hides dozens of engineering decisions.”

That is why **Requirement Engineering is critical.**

How could AI help in this example?

Possible answers could be:

- extract patient data automatically
- detect duplicate patient records
- analyze registration patterns
- suggest improvements to workflow

But we must understand that :

AI assists **after requirements are understood.**

Core Message

In the AI era, coding is becoming easier, but **understanding the problem correctly is becoming even more critical.**

Thus:

Requirement Engineering remains the **intellectual core of Software Engineering.**

If AI can generate code from requirements,

what becomes the most valuable skill for software engineers?

Answer:

Understanding the problem space.

We often confuse **what the system should achieve** with **how the system will achieve it**. The distinction among **Intent, Requirement, and Feature** resolves this confusion.

Let us take a Simple Problem

Suppose a university says:

“Students should be able to leave the campus quickly without long queues.”

Is this a **feature**, a **requirement**, or something else?

Our answers will create the learning moment.

Intent

Definition

Intent is the **underlying goal or purpose** that stakeholders want to achieve.

It expresses **why the system should exist**.

Intent is usually:

- high-level
- human-centered
- value-oriented
- not technical

Example:

“Students should leave the campus without unnecessary delays.”

This describes the **desired outcome**, not the system.

Characteristics of Intent

Intent is usually:

- vague
- broad
- sometimes emotional
- not directly implementable

Examples:

- improve student convenience
- reduce waiting time
- increase safety
- improve transparency

Intent answers the question:

Why should the system exist?

Requirement

Definition

A **requirement** translates intent into a **specific system capability or constraint**.

Requirements are:

- more precise
- testable
- system-oriented

Example:

The system shall allow students to generate a digital gate pass using their institutional login.

This requirement begins to describe **what the system must do**.

Characteristics of Requirements

Requirements must be:

- clear
- unambiguous
- verifiable
- consistent
- feasible

Requirements answer:

What must the system do to satisfy the intent?

Feature

Definition

A **feature** is the **actual functionality implemented in the system**.

Features are:

- design decisions
- specific implementations
- visible system capabilities

Example:

- QR-code based digital gate pass
- mobile app gate pass generation
- automated gate scanning

Features answer:

How will the system implement the requirement?

The Relationship

We can have this simple structure:

Intent
↓
Requirement
↓
Feature

Intent defines **purpose**.

Requirement defines **system capability needed to achieve the purpose**.

Feature defines **the implemented solution**.

Concrete Example

Example: Food Delivery Platform

Intent

Customers should receive food faster.

Requirement

The system shall allow users to track delivery status in real time.

Feature

- GPS-based delivery tracking
- live map display
- estimated arrival time

Example (Hospital System)

Intent

Doctors should access patient information quickly.

Requirement

The system shall allow doctors to retrieve patient medical history within 2 seconds.

Feature

- indexed medical database
- quick search interface
- patient dashboard

Why This Distinction Is Important

When teams confuse these levels, problems arise.

Example mistake:

Stakeholder says:

“We need a mobile app.”

But that may actually be a **feature**, not the real requirement.

The real intent might be:

“Patients should be able to access services remotely.”

Possible solutions may include:

- mobile app
- web portal
- telemedicine kiosk

So jumping to features too early can **hide the real problem**.

The Most Common Software Engineering Mistake

Many software teams jump directly from **intent** → **feature**, skipping requirement analysis.

Example:

Intent:

“Improve customer support.”

Feature implemented:

Chatbot.

But maybe the real requirement was:

Faster human response time.

Chatbot may not solve that.

Where AI Changes the Picture

AI is very good at generating **features quickly**.

Example:

- AI generates UI
- AI generates APIs
- AI generates code

But AI may not understand **intent properly**.

So the danger is:

Intent → AI-generated Feature

without properly defining requirements.

This leads to **misaligned systems**.

The Software Engineer's Role in the AI Era

Let us understand this clearly:

In the AI era, engineers must become experts in translating **intent into well-defined requirements.**

Because once requirements are clear:

AI can help generate

- design
- code
- tests
- documentation

But AI struggles when intent is vague.

Connecting to CSO Framework

This fits perfectly into our **Concept–Strategy–Operation** model.

CSO Layer	Meaning
Concept	Intent
Strategy	Requirements
Operation	Features

Intent expresses **purpose**.

Requirements define **system capabilities**.

Features implement **operational behavior**.

Classroom Exercise

Let us take this statement:

“Students should feel safer on campus at night.”

Please identify:

- 1.Intent
- 2.Requirements
- 3.Possible Features

Example answer:

Intent

Improve student safety

Requirement

Emergency reporting system

Feature

Mobile panic button

Software engineering fails when teams implement features without understanding intent.

We Should Remember

Intent explains why.

Requirements explain what.

Features explain how.

If we understand Intent, Requirements, and Features, how do we decide
what to build first?

This naturally leads to:

MVP (Minimum Viable Product)

What is MVP (Elementary Definition)

We can define it simply:

MVP is the smallest version of a product that delivers core value to users and allows learning through real-world feedback.

MVP = Minimum features needed to validate the core intent.

Connecting MVP with Intent–Requirement–Feature

This is the most important conceptual bridge.

Intent → Requirements → Features → Product

Intent drives MVP

MVP must satisfy the **core intent**, not all intents.

Requirements guide MVP

Only **critical requirements** are selected.

Features implement MVP

Only **essential features** are built.

The Key Idea

We can say:

“MVP is not about building less.

It is about building **only what is necessary to validate the intent.**”

Thank You

Minimum Viable Product & Various Types of Requirements

Anil Mokhade 18,20, and 24 March 2026

Story: *The Smart Campus Safety System*

Part 1 — The Intent (Start Here)

“Let me tell you a story from a campus like ours.”

There was growing concern among students:

“We don’t feel safe walking across campus at night.”

The administration took this seriously.

So they called a meeting.

Everyone agreed:

“We need a Smart Campus Safety System.”

What is this? Intent, Requirement, or Feature?

(confusion stays?)

Part 2 — The Jump to Features (The Mistake)

The IT enthusiast teams immediately started proposing solutions:

- Mobile safety app
- GPS tracking
- Panic button
- CCTV integration
- AI-based facial recognition
- Drone surveillance

Within a month, they had built a **feature-rich system**.

Very impressive.

- 12 modules
- real-time tracking
- AI alerts
- dashboards

They launched it.

Part 3 — The Failure

After 2 months:

- Students were not using it
- Many had not even installed the app
- Some found it intrusive
- Others didn’t trust the system

The project was technically successful.

But practically?

- **Failure.**

What went wrong?

Part 4 — The Real Problem

The team never clearly understood:

- When do students feel unsafe?
- Which locations are risky?
- What kind of help do they expect?
- Do they want surveillance or assistance?

The **intent was emotional and contextual**, but the team jumped to **features**.

Part 5 — Reframing Through Requirements

A new team was formed.

This time they started differently.

They spoke to students.

They observed behavior.

They analyzed complaints.

They discovered:

- Most issues occur between hostel and library
- Students feel unsafe when walking alone
- They don’t want surveillance — they want quick help

Now requirements became clearer:

- Provide quick access to help
- Reduce response time
- Ensure privacy

Part 6 — The MVP Thinking

Now they asked a different question:
“What is the **minimum system** we can build to test this?”

Instead of 12 features, they built:

MVP

- A simple mobile panic button
- Linked to campus security
- Sends location instantly
- Security responds within minutes

That’s it.

No AI. No drones. No complex dashboards.

Part 7 — The Result

After deployment:

- Students started using it
- Security response improved
- Trust increased

Then gradually they added:

- location heatmaps
- night patrol optimization
- optional tracking

Now the system evolved.

Part 8 — What is the Concept

“In the first attempt, we built many features without understanding intent.

In the second attempt, we built a Minimum Viable Product aligned with the core requirement.”

Critical Reflection (Very Important)

Insight 1

MVP is not about building less.
It is about building **what matters first**.

Insight 2

Features do not guarantee value.
Alignment with intent does.

Insight 3

Requirements are the **bridge** between intent and implementation.

AI Into the Story

“Now imagine this same situation in today’s AI world.”

The first team could have built even faster:

- AI-based surveillance
- automated alerts
- predictive analytics

In fact, they could build **more features in less time**.

But the problem would remain.

They still did not understand the intent.

AI-Era Insight

“AI reduces the cost of building features.
But it does not reduce the cost of misunderstanding the problem.”

Deeper Reflection

In AI systems, MVP must now validate:

- correctness of outputs
- trustworthiness
- ethical impact
- user acceptance

So MVP becomes:

Minimum Viable Understanding

(not just product)

Conclusion from the Story

“Software engineering fails not because we cannot build systems.

It fails because we build systems without understanding what truly matters.

MVP is our way of testing whether we have understood the problem correctly.”

Intent tells us why.

Requirements tell us what.

MVP tells us what to build first.

Features tell us how.

Google Glass Story

Introduction

“Around 2013, one of the most powerful technology companies in the world, Google, launched what they believed could be the future of computing.”

They introduced:

Google Glass

A wearable device:

- camera
- display in front of the eye
- voice commands
- internet connectivity

It looked futuristic.

It looked revolutionary.

What Was the Vision (Intent)?

The intent was:

“Provide seamless, hands-free access to information in real time.”

More deeply:

- augment human perception
- reduce dependence on phones
- enable instant interaction with digital world

What Were the Features?

The product included:

- voice commands (“OK Glass...”)
- photo/video capture
- navigation
- notifications
- augmented display

From an engineering perspective:

- Very impressive.

Was It an MVP?

Yes — in a way.

Google released it early as:

- “**Explorer Edition**”

- limited users
- high price (~\$1500)
- early adoption model

So technically:

- It was an MVP.

What Happened?

Despite technical brilliance:

- Public backlash started
- Privacy concerns grew
- People felt uncomfortable being recorded
- Some places banned it (restaurants, cinemas)
- Users felt awkward wearing it

A new term emerged:

- “**Glasshole**” (for users wearing it in public)

Finally:

- Google discontinued the consumer version.

Critical Analysis

The real learning for us:

What Went Wrong?

1. Intent Misalignment

Google’s intent:

Seamless digital augmentation

User’s perception:

Surveillance device

- Intent was not aligned with **social acceptance**

Requirement Failure

Key requirements were missing or misunderstood:

- privacy expectations
- social acceptability
- user comfort
- cultural context

These were **non-functional requirements**.

Feature-Driven Thinking

Google focused on:

- technology capabilities
- innovative features

But not enough on:

- human context
- user behavior
- emotional acceptance

MVP Misinterpretation

Google treated MVP as:

- “Early release of a feature-rich product”

But MVP should have been:

- “Test of core intent in real-world context”

Link to Intent–Requirement–Feature

Intent → Augment human experience

Requirement → Acceptable, safe, comfortable wearable interaction

Feature → Camera + display + voice interface

Problem:

- Jump from Intent → Feature
- Skipping deep Requirement understanding

The Real Failure

“Google Glass did not fail because of technology.

It failed because of misunderstanding **human and societal requirements.**”

If Google Glass Was Built Today

AI could make it even more powerful:

- real-time object recognition
- live translation
- facial recognition
- context-aware assistance

But ask students:

- Would that solve the core problem?

AI-Era Insight

“AI can amplify features.

But it cannot correct misunderstanding of intent.”

Deeper Insight

In AI systems, MVP must test:

- trust
- ethics
- privacy
- human comfort
- societal acceptance

Not just functionality.

Final Takeaways We can Have:

1. MVP is not about releasing early.

- It is about validating **core assumptions**.

2. Features do not guarantee success.

- Alignment with **intent and context** does.

3. Non-functional requirements can kill a product.

4. AI makes it easier to build.

- But easier to build the **wrong thing faster**.

Overall: Google Glass was a powerful product, but a weak understanding of human requirements. That is why it failed.

The Ideal Approach to Deciding MVP

First, correct a common misconception:

MVP is **not** “build the smallest thing.”

MVP is **build the smallest thing that validates the most important assumption.**

Start With the Right Question

What are we trying to learn?

Not:

- What can we build?
- What features should we include?

But:

- **What uncertainty must we reduce?**

Why “What are we trying to learn?” matters

Traditionally, teams ask:

- What features should we build?
- What system should we design?

But MVP thinking—and especially in the AI era—starts with:

What do we NOT know yet?

Because:

- Software projects fail due to **unknowns**, not **knowns**.

- MVP is not a building strategy.

It is a **learning strategy under uncertainty**.

It means identifying:

- **The biggest uncertainty in our understanding**

This uncertainty can be about:

1. Problem (Intent)

- Do users actually have this problem?
- Is this problem important?

2. Requirement

- What exactly do users need?
- Which requirement matters most?

3. Solution

- Will this solution work?
- Will users accept it?

4. Behavior

- How will users interact with the system?
- Will they trust it?

Types of Learning MVP Can Target

Type 1 — Problem Validation

- Are we solving the right problem?

Example:

Campus hospital:

Are long queues really due to registration?

Type 2 — Requirement Validation

- Which requirement matters most?

Example:

- faster registration
- better scheduling
- more doctors

Which one impacts waiting time?

Type 3 — Solution Validation

- Does our approach work?

Example:

Will digital registration actually reduce waiting time?

Type 4 — User Behavior Validation

- Will users use it?

Example:

Will students adopt a digital system?

Type 5 — AI Behavior Validation

- Does AI behave correctly?

Example:

- Is the chatbot accurate?
- Does it hallucinate?
- Is it trustworthy?

A Concrete Example

Example: Campus Safety System

Intent:

Improve safety at night

What are we trying to learn?

- Do students feel unsafe?
- Where do they feel unsafe?
- Do they want tracking or quick help?
- Will they use a safety app?

MVP Decision Based on Learning

If uncertainty is:

- “Will students use a panic button?”

Then MVP =

- simple panic button
- no complex system

“Earlier we built systems to deliver features.
Now we build systems to **answer questions**.”

Why This is Critical in AI Era

AI makes it easy to build:

- apps
- interfaces
- models

So the bottleneck shifts to:

- **understanding the problem correctly**

AI Era Insight

“When building becomes cheap, learning becomes expensive.”

What is Connection to Requirements Engineering

Requirement Engineering is about:

- understanding problem
- clarifying needs
- reducing ambiguity

MVP is:

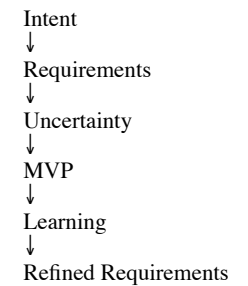
- **testing that understanding**

“We do not build MVPs to launch products.

We build MVPs to learn whether our understanding is correct.”

“A team that knows what to build may still fail.

A team that knows what it needs to learn will eventually succeed.”



Clearly Define Intent

Identify the **core purpose**.

Example:

“Reduce patient waiting time in campus hospital”

Important:

- There may be multiple intents
- You must select the **primary intent**

Critical Insight

MVP should validate **one core intent**, not all intents.

Identify Key Assumptions

Every system is built on assumptions.

- Do users actually face this problem?
- Will users use the system?
- Will this solution solve the problem?

Example assumptions:

- Students are delayed due to manual registration
- Digital registration will reduce waiting time

Critical Insight

MVP exists to **test assumptions**, not to deliver completeness.

Derive Minimum Requirements

From intent + assumptions, derive:

- **essential requirements only**

Not all requirements.

Example:

Instead of:

- full hospital system
- billing
- EMR
- analytics

Focus on:

- digital registration
- queue tracking

Critical Insight

MVP includes **only requirements needed to validate the core hypothesis**

Mapping Requirements to Minimal Features

- What is the **simplest implementation**?

Example:

Requirement:

Allow patient registration

Possible features:

- web form
- mobile app
- kiosk
- QR-based entry

MVP choice:

- simplest workable option

Critical Insight

MVP features are chosen for **learning efficiency**, not technological sophistication.

Define Success Criteria

This is often ignored but extremely important.

- How will we know MVP is successful?

Examples:

- average waiting time reduced by 30%
- 60% of patients use digital system
- registration time reduced below 2 minutes

Critical Insight

MVP without measurable success criteria is just a prototype.

Build–Measure–Learn Loop

This is where Agile comes in.

Build MVP

↓

Deploy

↓

Measure usage & behavior

↓

Learn

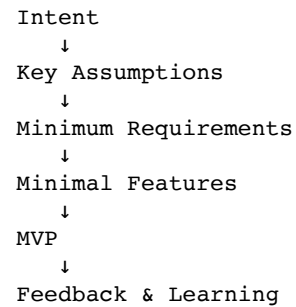
↓

Refine requirements

Critical Insight

MVP is not the end—it is the **beginning of learning**.

Putting It All Together



Common Wrong Approaches (Very Important)

Mistake 1 — Feature-Driven MVP

“Let’s build these features and call it MVP”
- Wrong direction

Mistake 2 — Technology-Driven MVP

“Let’s use AI, blockchain, etc.”
- Solution before problem

Mistake 3 — Too Small MVP

MVP so minimal that it delivers no value
- No learning possible

Mistake 4 — Too Big MVP

- Mini full product
- defeats purpose

MVP in AI Era (Critical Discussion)

TAKING AI context.

What AI Changes

- rapid prototyping
- faster feature development
- automated code generation

What AI Does NOT Change

- understanding intent
- identifying assumptions
- defining success criteria
- stakeholder alignment

New AI-Era MVP Insight

MVP must now validate not just functionality, but also:

- correctness
- trust
- explainability
- bias
- user acceptance

Example

AI chatbot MVP:

Not just:

- chatbot works

But:

- answers are correct
- users trust it
- no harmful outputs

“MVP is not a product decision.
It is a learning strategy.”

“In modern software engineering, especially with AI, the challenge is not building systems.
The challenge is deciding **what not to build first.**”

MVP Decision Framework

- 1.What is the **core intent**?
- 2.What is the **key assumption**?
- 3.What is the **minimum requirement** to test it?
- 4.What is the **simplest feature** to implement it?
- 5.How will we **measure success**?

Exercise

Intent:

Improve attendance in morning exercise sessions

- identify assumptions
- define MVP
- propose features
- define success criteria

MVP is Selection — But What Are We Selecting?”

Then subtitle:

Understanding Types of Requirements

Reflection on MVP

“When we decide an MVP, what exactly are we selecting?”

- features?
- functions?
- modules?

“Are we selecting features... or are we selecting something deeper?”

“We are actually selecting a **subset of requirements.**”

Exposing the Hidden Complexity

-Here is the real challenge:

Not all requirements are of the same type.

simple contrast:

- Some requirements define **what the system should do**
- Some define **how well it should perform**
- Some define **why the system exists**
- Some define **constraints and context**

If all these are different, can we treat them in the same way while designing an MVP?

No

Problem Revealed:

the core issue:

- ‘If we do not distinguish between types of requirements, we may build an MVP that is technically correct but practically useless.’

What do we mean by this?

What Does “Technically Correct” Mean?

A system is technically correct if:

- all **functional requirements** are implemented
- code works as expected
- inputs produce correct outputs
- no major bugs

Example:

- ✓ Patient registration works
- ✓ Data is stored
- ✓ IDs are generated

From a programmer’s perspective: **Success**

What Does “Practically Useless” Mean?

A system is practically useless if:

- users don’t adopt it
- it doesn’t solve the real problem
- it fails in real-world conditions
- it violates constraints (performance, usability, trust, etc.)

From a system / stakeholder perspective: **Failure**

Why This Happens

Because engineers focus only on:

Functional Requirements (what system does)

and ignore:

- Non-functional requirements
- Stakeholder requirements
- Business requirements
- Contextual constraints

Example 1 — Campus Hospital MVP Built (Technically Correct)

System supports:

- patient registration
- token generation

Everything works perfectly.

What Was Ignored

Non-Functional Requirement

- system is slow (5–6 seconds per entry)

Usability Requirement

- interface is complex
- staff struggle to use it

Operational Requirement

- system crashes during peak hours

Result

- queues remain
- staff revert to manual system

Technically correct, practically useless

Example 2 — Food Delivery App Functional MVP

- browse restaurants
- place order
- make payment

All features work.

Missing Requirements

Performance Requirement

- app takes 10 seconds to load

Trust Requirement

- delivery time inaccurate

UX Requirement

- confusing interface

Result

- users uninstall app
- System works → Product fails

Example 3 — AI Chatbot

Functional MVP

- chatbot answers questions
- uses LLM
- responds fluently

Missing Requirements

Accuracy Requirement

- answers sometimes incorrect

Trust Requirement

- users don't rely on answers

Safety Requirement

- harmful responses possible

Result

- users stop using chatbot
- system is withdrawn

AI works → System fails

Example 4 — Attendance System

Functional MVP

- biometric attendance system
- captures entry time

Missing Requirements

Context Requirement

- students queue for attendance

Performance Requirement

- slow scanning

Behavioral Requirement

- students find ways to bypass system

Result

- chaos increases
- attendance data unreliable

System works → Problem worsens

Deeper Analytical Insight a layered failure model:

Intent (Why)



Requirements (What)



Features (How)

Failure occurs when:

- intent is misunderstood
- requirement types are incomplete
- features solve wrong subset

The Core Mistake

Engineers assume:

Functional Requirements = Complete System

But in reality:

System = Functional + Non-functional + Context + Stakeholder + Constraints

Why MVP Suffers the Most

MVP is **minimal by design**. So if you choose:

wrong requirement types

you amplify the error.

Example

If MVP includes:

✓ Functional requirement

✗ (not) Non-functional requirement

Then:

MVP fails immediately in real-world use

AI Era Amplification

This becomes even more critical in AI systems.

AI System Can Be:

- ✓ Functionally correct
- ✓ Technically advanced

But fail due to:

- bias
- hallucination
- lack of explainability
- lack of trust

Example

AI diagnosis system:

- ✓ predicts disease
- ✗ doctor cannot understand reasoning

Not adopted

Key Engineering Principle

We can state this strongly:

“A system that satisfies only functional requirements is incomplete.”

If you ignore non-functional requirements, the system may work.

If you ignore stakeholder requirements, the system may not be used.

If you ignore business requirements, the system may not matter.

How to Avoid This Problem

We can use this checklist:

Before finalizing MVP, ask:

1. Have we addressed **core intent**?
2. Have we selected **critical functional requirements**?
3. Have we included **minimum non-functional requirements**?
4. Have we considered **user behavior and context**?
5. Have we defined **success criteria**?

Final Synthesis

We can conclude:

“Software Engineering is not about building correct systems.
It is about building systems that are **correct in context**.”

**A system that works is not enough.
A system must work *for the problem*.**

Back to the Classification of Requirements:

Example

Campus Hospital

Intent:

Reduce waiting time

Case 1 — Only Functional Thinking

MVP includes:

- patient registration
- token generation

But ignores:

- performance (slow system)
- usability (complex UI)

Result:

System exists, but does not solve problem.

Case 3 — Ignoring Business Requirement

System works perfectly but:

- does not reduce waiting time

Result:

Wrong MVP

So the real problem is not selecting MVP.

The real problem is selecting the **right types of requirements for MVP**.

Case 2 — Ignoring Stakeholder Requirements

MVP works for admin, but:

- doctors cannot access data easily
- patients find system confusing

Result:

Poor adoption

We can explicitly connect:

Intent

↓

Different Types of Requirements

↓

MVP Selection

To design a correct MVP, we must first understand **what kinds of requirements exist**.

The Central Question

What kinds of requirements do we need to consider as software engineers?

- functional
- non-functional

Is that enough?

Introducing Classification as a Need

Requirement classification is not a theoretical exercise.

It is a practical necessity to ensure that we do not miss critical aspects of the system.

Connecting Back to MVP

This is very important.

MVP is essentially a **selection problem**.

And you cannot select correctly unless you understand **what you are selecting from**.

The Transition Line Classification to Selection:

So far, we have understood how to decide *what to build first* using MVP.

Now we must understand *what constitutes a requirement* in the first place.

What is Classification

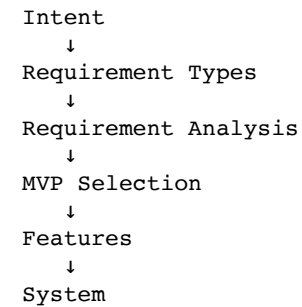
We classify requirements so that we can analyze problems systematically.

- User requirements
- Functional requirements
- Non-functional requirements

And extended beyond.

Requirement classification is essentially a way of structuring **problem understanding**.

Integrated Requirement Understanding Process



Classical Classes of Requirements

Most textbooks (e.g., Software Engineering, Software Requirements) classify requirements into:

1. User Requirements

High-level, natural language statements describing what users need.

Example:

“Students should be able to register at the campus hospital easily.”

Characteristics:

- informal
- understandable by non-technical stakeholders
- may be ambiguous

Functional Requirements

Define **what the system should do**.

Example:

“The system shall allow patients to register by entering personal and medical details.”

Characteristics:

- specific system behavior
- input–output oriented
- testable

Non-Functional Requirements (NFRs)

Define **how well the system performs its functions**.

Examples:

- “Registration should complete within 2 seconds.”
- “System should support 1000 concurrent users.”
- “Patient data must be encrypted.”

Categories of NFRs:

- performance
- security
- usability
- reliability
- scalability

Textbook View

User Requirements

↓

Functional Requirements

+

Non-Functional Requirements

Are These Sufficient?

This classification is **necessary but not sufficient** for real-world systems.

Why It Is Not Sufficient

Because real systems involve:

1. Multiple Stakeholders

- users
- administrators
- regulators
- maintainers
- business owners

Each has different requirements.

System Context

- integration with other systems
- organizational policies
- hardware constraints

AI-Based Systems (Very Important)

In AI systems, we need additional concerns:

- model accuracy
- bias
- explainability
- data requirements
- continuous learning

Lifecycle Considerations

- deployment
- maintenance
- upgrades
- evolution

Critical Insight

Functional + Non-functional requirements describe the system, but they do not fully capture the **context in which the system operates**.

Why We Need More Classes

Because real systems are:

- socio-technical
- multi-stakeholder
- regulated
- evolving
- data-driven (especially in AI systems)

So we need a **broader classification**.

Expanded Classes of Requirements

Business Requirements

Define **why** the system exists

Example:

- “Reduce hospital waiting time by 30%”
- “Increase online consultation usage by 50%”

✓ Connects to **intent and value**

Stakeholder Requirements

Needs of different stakeholders

Example (Hospital):

- Doctor → quick access to patient history
- Patient → shorter waiting time
- Admin → efficient billing

✓ Often conflicting → requires negotiation

System / Technical Requirements

Constraints on implementation

Examples:

- must run on cloud infrastructure
- must use specific database
- must support REST APIs

✓ Affect architecture decisions

Interface Requirements

Interaction with external systems

Examples:

- integration with payment gateway
- integration with student database
- API compatibility

✓ Critical for real-world systems

Domain Requirements

Requirements arising from **application domain**

Example:

- medical diagnosis protocol
- banking transaction rules
- railway signaling constraints
- ✓ Not invented by developer
- ✓ Must be **understood from domain knowledge**

Data Requirements

What data is needed and how it is handled

Examples:

- patient data schema
- data retention policy
- backup frequency
- ✓ Very important in AI systems

Regulatory / Legal Requirements

Compliance with laws and policies

Examples:

- healthcare privacy regulations
- GDPR-like data protection
- audit requirements
- ✓ Ignoring these can make system unusable

Organizational Requirements

Constraints imposed by organization

Examples:

- must follow institute policies
- must integrate with legacy systems
- must use approved vendors

Operational Requirements

Real-world system operation

Examples:

- 24x7 availability
- disaster recovery
- maintenance window

✓ Often ignored in student projects

AI-Specific Requirements (Modern Addition)

This is where your course becomes **cutting-edge**.

Examples:

- model accuracy $\geq 90\%$
- bias detection and mitigation
- explainability of predictions
- training data quality
- continuous monitoring

✓ Not covered in classical SE textbooks
✓ Critical for modern systems

Quality Requirements (Extended NFRs)

Sometimes treated as a refined category of NFRs.

Examples:

- reliability (uptime 99.9%)
- usability
- maintainability
- scalability

Business Requirements (Why)



Stakeholder Requirements



User Requirements



Functional Requirements (What)



Non-Functional Requirements (How well)



System / Interface / Data / Regulatory / Operational



AI-specific Requirements (if applicable)

Critical Insight

“Functional and non-functional requirements describe system behavior.
But real systems are governed by many more constraints.”

If you miss a functional requirement, a feature may fail.
If you miss a non-functional requirement, the system may perform poorly.
But if you miss business, domain, or regulatory requirements, the system may be irrelevant or illegal.

What is the difference between Intent & User Requirements?

The Core Difference

Intent = Why the system should exist

User Requirement = What the user expects the system to do

Conceptual Separation

Aspect	Intent	User Requirements
Nature	Purpose / Goal	Expectation / Need
Level	Abstract	Semi-concrete
Focus	Problem	Solution (partial)
Stability	More stable	More changeable
Source	Stakeholders / organization	Users / stakeholders
Expression	Often implicit	Usually expressed

Deep Meaning

Intent

Represents the **core motivation**

- Why are we building this system?
- What problem are we trying to solve?
- What value do we want to create?

- ✓ Often **not clearly articulated**
- ✓ Often **hidden behind requirements**

Example (Campus Hospital)

Intent:

Reduce patient waiting time

Critical Insight

User requirements are attempts to express intent—but they may not fully capture it.

User Requirements

Represent **user expectations of the system**

- What should the system provide?
- What functionality should exist?

Example

User requirements:

- System should allow patient registration
- System should generate tokens
- System should display queue status

Where the Confusion Happens

New S/w Developers often assume:

User Requirement = Problem

But actually:

User Requirement = One interpretation of the problem

Example1 Where They Diverge

Scenario: Hospital Queue

Intent

Reduce waiting time

User Requirement (stated)

“We need a faster registration system”

Hidden Issue

Actual problem may be:

- uneven doctor availability
- poor scheduling
- emergency interruptions

Result

System built:

- ✓ fast registration
- ✗ waiting time unchanged

Intent not achieved

Key Engineering Principle

Intent defines correctness.

User requirements define implementation direction.

Example 2

Scenario: Students Attendance

Intent

Improve attendance

User Requirement

“We need biometric attendance system”

Reality

Problem may be:

- lack of interest
- teaching quality
- scheduling issues

Result

- ✓ biometric system works
- ✗ attendance unchanged
- Wrong solution for right problem

Relationship Between Them

Intent (Why)

↓

User Requirements (What users think they need)

↓

Functional Requirements (System behavior)

↓

Features (Implementation)

Why This Distinction Matters

If you ignore intent:

- you build wrong system
- MVP fails
- product fails

If you ignore user requirements:

- system may not be usable
- stakeholders unhappy

AI Era Insight

This distinction becomes even more important now.

AI can:

- generate user requirements
- convert them into features
- build systems

But AI cannot fully:

- understand intent
- resolve conflicting goals
- interpret context

So:

Human role shifts from writing requirements to clarifying intent

“Users tell you what they want.

Engineering begins when you ask why they want it.”

Exercise

“A user says: ‘We need a dashboard.’

What could be the intent?”

- monitoring?
- decision making?
- reporting?
- control?

“Dashboard is NOT a requirement.

It is a **solution assumption.**”

1. Monitoring

Intent:

- Observe system state in real-time
- Track performance

Example:

- hospital patient queue
- server health

Implication

Dashboard should show:

- live data
- alerts
- trends

2. Decision Making

Intent:

- support human decisions

Example:

- which doctor to assign
- which patient to prioritize

Implication

Dashboard should include:

- analytics
- recommendations
- summaries

3. Reporting

Intent:

- communicate information
- generate summaries

Example:

- daily hospital report
- attendance report

Implication

Dashboard should include:

- historical data
- export features
- structured summaries

4. Control

Intent:

- take action on system

Example:

- assign doctor
- change schedule
- trigger alerts

Implication

Dashboard becomes: not just display, but **interactive control interface**

Critical Insight

Same “dashboard” → completely different systems.

If intent = Monitoring

simple visualization system

If intent = Decision Making

analytics + insights system

If intent = Control

command + execution system

If intent = Reporting

documentation system

The Core Engineering Lesson

“Same requirement statement can correspond to different intents.”

Why This Matters

If intent is misunderstood:

Case 1

Intent = decision making

But system built = monitoring dashboard

No real value

Case 2

Intent = control

But system built = reporting dashboard

System useless in operations

“We cannot design the system unless we clarify the intent.”

Proper Engineering Response

When user says:

“We need a dashboard”

Engineer should ask:

- What decisions will this support?
- What actions should users take?
- What information is critical?
- How frequently is data needed?
- Who are the users?

Question:

“A user says: ‘We need a dashboard.’ What could be the intent?”

Answer:

Possible intents include:

- Monitoring system state
- Supporting decision making
- Reporting information
- Controlling system operations

However:

The statement “we need a dashboard” is not a requirement but a **solution expression**.

The actual intent must be elicited through further questioning.

When a user asks for a dashboard,
they are not asking for a screen—they are asking for **clarity, control, or confidence**.

*“Dashboard is not the requirement.
Understanding why the dashboard is needed is the requirement.”*

Before accepting any requirement, you should ask:

- 1.What is the **intent behind this requirement**?
- 2.Does this requirement **directly support the intent**?
- 3.Are there **better ways to satisfy the intent**?

Intent is the truth of the problem.

User requirements are interpretations of that truth.

Intent tells us why to build.

User requirements tell us what to build.

Confusing them leads to building the wrong system correctly.

Requirements Engineering is the disciplined process of understanding the right problem before building the solution.

Final integrated flow:

Intent (Why)
↓
Stakeholders & Context
↓
Requirement Elicitation
↓
Requirement Analysis
↓
Requirement Classification
↓
Specification
↓
Validation
↓
Management (Evolution)
↓
MVP & Iterative Development
↓
Learning & Refinement

Intent is the Foundation

- Every system begins with a **purpose**
 - Users often express **solutions, not intent**
- Engineering begins by asking “**Why?**”

Requirements Have Multiple Types

Not just:

- user
- functional
- non-functional

But also:

- business
- stakeholder
- domain
- data
- regulatory
- operational
- AI-specific

Missing any type → system failure

Requirements are Structured Understanding

Requirements are not:

- wishes
- features
- UI elements

They are:

interpreted, refined, and negotiated understanding of the problem

Requirements Engineering is a Continuous Process

Not a one-time phase.

Includes:

- elicitation
- analysis
- specification
- validation
- management

Iterative and evolving (Agile)

MVP is Learning Through Requirements

- MVP is not minimal product
- It is minimal system to **test understanding**

Build → Measure → Learn → Refine requirements

Common Mistakes (Very Important).

✗ Mistake 1

Confusing:

Requirement = Feature

✗ Mistake 2

Ignoring:

- non-functional requirements
- stakeholder diversity
- domain constraints

✗ Mistake 3

Assuming:

Requirements are known upfront

✗ Mistake 4

Over-relying on AI

Role of AI in Requirement Understanding

AI can:

- summarize requirements
- generate user stories
- detect inconsistencies

But cannot fully:

- understand intent
- resolve conflicts
- handle socio-technical complexity

AI **augments**, not replaces, Requirements Engineering

Deep Insight

“The hardest problem in software is not building the system.
It is understanding what system should be built.”

A Practical Checklist for Software Developers

Before building any system, ask:

1. What is the **intent**?
2. Who are the **stakeholders**?
3. What are the **different types of requirements**?
4. What are the **assumptions**?
5. What is the **MVP to test them**?
6. How will we **validate success**?

*“Software Engineering is not about writing
correct code.*

*It is about building systems that are correct in
purpose, context, and impact.”*

Requirement Understanding =

- Intent (Why)
- Stakeholders (Who)
- Requirements (What)
- Constraints (Under what conditions)
- Validation (How do we know it is correct)
- Evolution (How it changes over time)

Thank You

Software Design - Component Design with Product Perspective

Anil Mokhade 14 April 2026

“If architecture is decided, is the software ready?”

Answer Could be : Almost / Yes / Not fully?

But actually:

“Architecture tells us **what exists**.

But it does NOT tell us **how it actually works**.”

RECALL ARCHITECTURE - Let us CONNECT BACK:

Architecture defines:

- System structure
- Major components
- Interaction patterns
- Technology choices

Example: “We said: Web app + Backend + Database + AI module”

But :

“What is inside this ‘AI module’?”

What exactly is ‘Backend’?

How does ‘Booking’ actually happen?”

“Architecture is a map.

Component Design is the actual construction.”

THE GAP (CRITICAL INSIGHT)

Architecture → ??? → Code

“What is missing here?”

Component Design fills this gap

FORMAL TRANSITION DEFINITION

Component Design is where architecture becomes executable reality.

Or more powerfully:

“Architecture is intention.

Component Design is realization.”

CSO-BASED TRANSITION

CONCEPT SHIFT

Architecture	Component Design
What exists	What it does
Structure	Behavior
Boxes & arrows	Logic & data
Static	Dynamic

OPERATION SHIFT

Architecture	Component
High-level deployment	Actual execution
System-level scaling	Performance tuning
System monitoring	Component-level logging

RUNNING EXAMPLE

Example: Hospital Appointment System

Architecture View:

- Frontend
- Backend
- Database
- AI Chatbot

“What is ‘Backend’?”

Can we answer precisely?

Backend → broken into:

- Appointment Component
- Doctor Availability Component
- Notification Component
- Payment Component

Appointment Component

- What input does it take?
- What validation is needed?
- What data structure?
- What algorithm?
- What errors?

So,

“Now we are doing Component Design.”

We designed microservices—but one service calls another synchronously 10 times → system slows down.”

This is **component-level mistake, not architecture mistake**

AI ERA TRANSITION (CRITICAL ADDITION)

“Earlier components were deterministic.
Now what if one component is an LLM?”

New complexity:

- Output not fixed
- Behavior not predictable
- Needs evaluation, not just testing

Transition Statement:

“**In AI systems, component design is no longer just coding—it is designing behavior under uncertainty.**”

AGILE PERSPECTIVE TRANSITION

“When do we design components in Agile?”

“Before coding”?

Actually,

“We design components **inside each sprint.**”

- Each user story → leads to component refinement
- Design evolves
- Refactoring = continuous design

PRODUCT THINKING TRANSITION

“Does the user care about architecture?”

Answer: **✗** No

“Does the user care about component behavior?”

Answer: **✓** YES

Example:

- User doesn’t care about MVC
- User cares:
 - Booking works fast
 - Suggestions are correct

Transition Line:

“**Product quality is experienced at the component level.**”

FINAL TRANSITION DIAGRAM

User Need
↓
Architecture (System Structure)
↓
Component Design (Behavior + Logic)
↓
Code (Implementation)
↓
Execution (User Experience)

Extended for AI:

Component = Code + Data + Model + Feedback

WHAT IS COMPONENT DESIGN? (Concept Layer – CSO)

Classical Definition (Refined)

Component Design is:

The process of transforming **architectural elements** into **implementable, testable, and evolvable units**.

But in **AI + Product Era**, refine it:

Component = A unit of capability that delivers a user-relevant function, with clear interfaces, behavior, and evolution strategy.

Shift in Thinking

Classical	Modern (AI + Product)
Code unit (class/module)	Capability unit (feature/service)
Static behavior	Adaptive / data-driven behavior
Designed once	Continuously evolving
Interface = API	Interface = API + Prompt + Data contract

Key Insight

Architecture answers: “What exists?”

Component Design answers: “How exactly will it work?”

COMPONENT DESIGN THROUGH CSO FRAMEWORK

CONCEPT (What is inside a component?)

Each component must define:

- Responsibility (single purpose)
- Inputs / Outputs
- Data it owns
- Behavior (logic or model)
- Constraints (performance, safety, AI risks)

Example (Hospital AI Chatbot Component)

Element	Example
Responsibility	Book appointment
Input	Natural language query
Output	Structured booking
Behavior	LLM + rule engine
Constraints	No hallucination

STRATEGY (How to design components?)

This is where **patterns + decisions** come.

Component Design Strategies

1. Decomposition Strategy

- Functional (login, booking)
- Domain-driven (patient, doctor, appointment)
- AI-driven (intent detection, response generation)

Data vs Behavior Strategy (AI impact)

- Classical: Logic dominates
 - AI Era: **Data + Model dominates**
- Example:
- Old: `if-else` logic
 - New: ML model predicts behavior

Coupling & Cohesion Strategy

Good Design	Bad Design
High cohesion	Mixed responsibilities
Low coupling	Tight dependencies

State vs Stateless Design

- Microservices → Stateless preferred
- AI systems → Stateful (context matters)

OPERATION (How it runs in real world?)

Now comes:

- Deployment
- Monitoring
- Scaling
- Feedback loops

AI Era Addition:

Component is incomplete without:

- Telemetry
- Feedback loop
- Model update mechanism

COMPONENT DESIGN PATTERNS (VERY IMPORTANT)

Classical Patterns

1. MVC (Model-View-Controller)

- Separation of concerns

2. Layered Pattern

- UI → Business → Data

3. Repository Pattern

- Central data management

Modern Distributed Patterns

4. Microservices Pattern

- Each component = independent service

5. Event-Driven (Pub-Sub)

- Loose coupling

6. API Gateway Pattern

- Single entry point

AI-Specific Patterns (NEW — Critical)

1. LLM Wrapper Pattern

- Encapsulate LLM calls inside a component
- Why?
- Avoid spreading prompts everywhere

2. Retrieval-Augmented Generation (RAG) Pattern

- Component = Retriever + Generator

3. Human-in-the-Loop Pattern

- Component requires human validation

Insight

“In AI systems, components are not just code—they are **learning entities**.”

4. Guardrail Pattern

- Filters outputs (safety, bias)

5. Model + Rule Hybrid Pattern

- AI + deterministic logic

6. Feedback Learning Component

- Improves over time using telemetry

COMPONENT DESIGN IN AGILE PROCESS

Vertical Slice Thinking (We discussed earlier)

Instead of building layers:

Build **end-to-end feature slices**

Example:

✗ Wrong:

- First DB
- Then API
- Then UI

✓ Correct:

- One feature → complete flow

Component Design in Sprint

Each sprint should produce:

- Working component
- Tested component
- Deployable component

Definition of Done (Component Level)

A component is done when:

- Function works
- Interfaces stable
- Tested (unit + integration)
- Observability added
- AI behavior validated (if applicable)

Refactoring as Design Activity

Agile insight:

Design is not done once—it is continuously refined.

PRODUCT PERSPECTIVE ON COMPONENT DESIGN

Core Shift

From:
“Is this component correct?”
To:
“Does this component create user value?”

Product-Oriented Component Questions

For each component, ask:
1.What user problem does it solve?
2.What metric defines its success?
3.What happens if it fails?
4.Can it evolve with user behavior?

Example: Recommendation Component

Product Metric	Design Impact
CTR (Click-through rate)	Model tuning
Response time	Caching
Personalization	Data pipeline

“A well-designed component is not one that is elegant,
but one that **survives real users.**”

AI ERA IMPACT ON COMPONENT DESIGN

New Types of Components

- Data pipeline components
- Model training components
- Prompt engineering components
- Evaluation components

New Risks

- Hallucination
- Model drift
- Bias
- Non-determinism

New Lifecycle

Component lifecycle now includes:
1.Design
2.Build
3.Deploy
4.Monitor
5.Retrain
6.Redeploy

Deep Insight

Classical:
Component = deterministic function
AI Era:
Component = probabilistic behavior system

INTEGRATED EXAMPLE

Problem:

Design component for
Campus Placement Predictor

Step 1: Identify Components

- Student Profile Component
- Skill Analysis Component
- Prediction Component (ML)
- Dashboard Component

Step 2: Apply Patterns

- Microservices
- RAG (for insights)
- LLM wrapper (for explanations)

Step 3: Define One Component (Prediction)

Element	Design
Input	Student data
Output	Placement probability
Behavior	ML model
Pattern	Model + API
Risk	Bias

Step 4: Agile Execution

- Sprint 1 → Basic prediction
- Sprint 2 → Improve accuracy
- Sprint 3 → Add explainability

Architecture → Components → Interfaces → Behavior → Evolution

Component = Code + Data + Model + Feedback

In the AI era, software design is no longer about building static systems—
it is about designing **living components that learn, adapt, and interact with humans.**

Interface Design
(API + Prompt Design + Data Contracts)

Why Interface Design is Critical

When we first see architecture diagrams, we often think:

“Components are defined — we are done.”

But in real systems, **failures rarely come from internal logic** — they come from **bad interfaces**.

A component without a well-defined interface is unusable, unsafe, and unmaintainable.

Conceptual Definition (CSO – Concept)

✓ What is an Interface?

An interface defines **how two entities communicate**.

It specifies:

- What can be requested
- What will be returned
- What assumptions are valid
- What errors may occur
- What guarantees are provided

Key idea:

An interface is not a function signature — it is a **contract of behavior**.

Interface includes:

- Function/API signatures
- Data formats and semantics
- Allowed operations
- Preconditions and postconditions
- Error conditions
- Performance and reliability guarantees
- Security constraints
- AI behavior constraints (in modern systems)

Strategy: Three Types of Interfaces in Modern Systems

Modern systems need **three categories** of interfaces:

Type	Purpose
API Interfaces	Component-to-component communication
Prompt Interfaces	Human-to-AI / Component-to-LLM communication
Data Contracts	Data exchange agreements

API Interface Design

What is an API Interface?

An API is an explicit way for one software component to use another.

Example:

```
POST /appointments
{
  "patient_id": "1234",
  "doctor_id": "5678",
  "date": "2026-03-20"
}
```

Core Principles of Good API Design

Principle	Meaning
Clarity	Function should be self-explanatory
Consistency	Naming conventions uniform
Stability	Avoid breaking changes
Versioning	Support backward compatibility
Explicitness	Avoid hidden side effects
Idempotency	Repeated requests behave the same
Error	Clear error messages

Poor API:

```
doStuff(data)
```

Good API:

```
scheduleAppointment
(patientID,
doctorID, timeslot)
```

API Contracts and Failure Modes

A strong API interface explicitly defines:

- Input format
- Output format
- Error responses
- Latency expectations
- Security rules
- Retry behavior

Example:

```
Status 400: Invalid input
Status 401: Unauthorized
Status 409: Appointment already exists
```

API Design in Agile

In Agile:

- APIs are designed **early**, but evolve.
- Consumer-driven contracts are used.
- Mock APIs allow parallel work.

Example workflow:

1. Define API contract
2. Frontend & backend work in parallel
3. Validate with tests
4. Iterate

Data Contract Design

What is a Data Contract?

A data contract defines:

“What data is exchanged, what it means, and what guarantees apply.”

Includes:

- Schema
- Semantics
- Ownership
- Validity constraints
- Evolution rules

Why Data Contracts Matter

Because:

- Misinterpreted data → system failure
- AI models amplify bad data
- Distributed teams depend on clarity

Example Data Contract

```
{
  "appointment_id":
    "string",
  "doctor_id": "string",
  "patient_id":
    "string",
  "timestamp":
    "ISO-8601",
  "status": "CONFIRMED |
CANCELLED"
}
```

Evolution Rules

Critical in real systems:

- Fields should not change meaning
- Additive changes preferred
- Deprecated fields handled gracefully

Prompt Design as an Interface (AI Era)

This is **new and essential** for AI-driven software.

What is a Prompt Interface?

A prompt is an **instructional interface** between:

- User ↔ LLM
- System ↔ LLM
- Component ↔ LLM

Example:

"You are a medical assistant. Summarize the patient's query into structured form."

Prompt = Behavioral API

Think of prompts as **behavior contracts**.

They define:

- Role
- Task
- Constraints
- Expected output format

Example Prompt Contract

Input:

User message

Output:

```
{
  "intent": "",
  "entities": [],
  "urgency": ""
}
```

Components of a Good Prompt Interface

Element	Example
Role	“You are a medical assistant”
Task	“Extract symptoms”
Context	“Patient history: ...”
Format	JSON
Constraints	“Do not give medical advice”

Prompt Versioning (Critical Concept)

Prompts evolve like APIs.

- v1: Basic instructions
- v2: Adds constraints
- v3: Adds safety rules

Treat prompts as **version-controlled artifacts**.

Interface Design for AI Components

AI introduces new challenges:

Issue	Impact
Non-determinism	Outputs vary
Drift	Behavior changes over time
Bias	Unfair responses
Safety risks	Harmful output possible

Therefore, AI Interface Contracts Must Include:

- Evaluation metrics
- Safety guardrails
- Fallback behavior
- Monitoring signals

Example AI Component Interface

Input: User Query

Output: Intent +
Confidence Score

If confidence < 0.7:
 fallback to human

Testing Interface Contracts

API Testing:

- Schema validation
- Error cases
- Performance tests

Data Contract Testing:

- Schema enforcement
- Compatibility checks

Prompt Testing:

- Behavioral tests
- Adversarial prompts
- Drift monitoring

Product Thinking Angle

Users don’t see APIs or prompts.

They experience:

- Reliability
- Speed
- Safety
- Consistency

So interface design directly affects:

- User trust
- Business success
- System scalability

Common Interface Failures

Failure	Cause
Tight coupling	Hidden dependencies
Silent failures	Poor error design
Hallucinations	Weak prompt contract
Data mismatch	Poor data contract
Fragile integrations	No versioning

Key Takeaway

In modern software systems, **interfaces define the real architecture**.

If interfaces are poorly designed:

- Good components still fail
- AI behaves unpredictably
- Systems become fragile

Thank You

1. Testing & Validation of Components and Interfaces

2. DevOps

Anil Mokhade 17 April 2026

Introduction

“We have designed software systems—components, interfaces, architectures.
But let me ask you a simple question...”

“How do we know that what we have designed will actually work?”

Cognitive Tension?

“A system may be logically correct...

A component may pass all tests...

An interface may follow all specifications...”

“And yet... the system may fail.”

“Software does not fail when we design it.
It fails when it interacts—with other components, with users, and with the real world.”

Key shift:

- From **structure** → **behavior**
- From **design correctness** → **interaction correctness**
- From **individual components** → **emergent system behavior**

This is where **Testing and Validation** come in.”

Let us define:

- **Testing** → checking whether components and interfaces behave as expected
- **Validation** → checking whether the system behaves as *needed in reality*

Link to Previous Lecture (Critical Continuity)

“In Software Design, we focused on *structure*.

Now, we shift our focus to *behavior*.”

Design → Structure

Testing & Validation → Behavior

Bridge to Systems Thinking

“In systems engineering, correctness is not just about parts—it is about interactions.”

- connects beautifully with our System thinking approach.

Anchor Question

“So the real question we explore today is:

How do we ensure that a designed system actually works—reliably, safely, and meaningfully?”

So We Look into

- Component Testing
- Interface Testing
- DevOps story

“And in the age of AI, where outputs are not always predictable... Testing becomes even more challenging—and more important.”

Conceptual Foundation (CSO Mapping)

CONCEPT

Testing & Validation ensures:

- Components behave as intended
- Interfaces communicate correctly
- System behavior emerges safely

STRATEGY

- Define **testable units**
- Validate **contracts (interfaces)**
- Simulate **real-world conditions**
- Include **AI-specific uncertainty handling**

OPERATION

- Unit tests, integration tests
- API testing, contract testing
- Simulation, telemetry, monitoring

Component Testing (Unit Level)

- Each component must be **independently testable**
- Isolation is critical → **mocking/stubbing**

[Component A] → uses → [Database/API]

Replace with:

[Component A] → uses → [Mock DB/API]

Key Idea:

“If you cannot test it independently, your design is flawed.”

Interface Testing (MOST CRITICAL IN MODERN SYSTEMS)

Expanding beyond traditional API:

We must include:

1. API Interfaces

- REST, GraphQL
- Input/output correctness
- Error handling

2. Data Contracts

- Schema validation (JSON/XML)
- Version compatibility

3. Prompt Interfaces (AI Era) (Very Important)

- Prompt → Response behavior
- Non-deterministic outputs

Integration Testing

Core idea:

“Most failures occur not inside components, but between them.”

Example:

- Payment service works ✓
- Order service works ✓
- But payment not linked to order ✗

We need to Learn:

- Sequence validation
- Timing issues
- State consistency

Validation vs Verification (Critical Distinction)

Very important for **product thinking**

Aspect	Verification	Validation
Question	Built right?	Built the right thing?
Focus	Design correctness	User needs
Example	API returns correct JSON	User understands output

AI Era Expansion

Problem: Traditional Testing Assumes Determinism

But AI systems:

- Non-deterministic
- Context-sensitive
- Probabilistic

New Testing Dimensions

1. Prompt Testing

- Same prompt → different outputs
- Sensitivity to wording

2. Output Validation

- Correctness is fuzzy
- Needs human-in-loop

3. Bias & Safety Testing

- Ethical risks
- Hallucination detection

Connect to our previous lecture:

- Anthropometric + behavioral requirements

Types of Testing (Reframed for our Course)

Instead of traditional list, let us learn like this:

Structural Testing

- Unit testing
- Code coverage

Behavioral Testing

- Integration testing
- Workflow validation

Contract Testing

- API + data schema validation

Experiential Testing

- UX validation
- Human interaction

AI Testing

- Prompt stability
- Output trustworthiness

Design Principles for Testability

Teach this as **reverse thinking from testing → design**

Key principles:

1. Loose Coupling

- Easier isolation

2. High Cohesion

- Clear functionality

3. Dependency Injection

- Replace real components with mocks

4. Clear Interfaces

- Explicit contracts

5. Observability

- Logs, metrics, traces

“Testability is a design property, not a testing activity.”

Agile + DevOps Perspective

Continuous Testing

- Happens in every sprint

CI/CD Pipelines

- Automated testing

Shift Left Testing

- Testing starts early (design phase)

Shift Right Testing

- Monitoring in production

Failure-Oriented Thinking

Always **Ask:**

- What if interface fails?
- What if data is delayed?
- What if response is inconsistent?

- Fault injection
- Chaos testing

Example

System: AI-Based Mentor Chatbot

Component Testing:

- NLP module
- Recommendation engine

Interface Testing:

- Prompt → Response
- Student query → AI output

Integration Testing:

- Chat UI + AI engine + database

Validation:

- Does student feel guided?

Does this connect with your existing project?

Some Questions to Ponder

1. Why do systems fail even when all components are correct?
2. Can an API be correct but still useless?
3. How do you test something that gives different outputs every time? (AI)
4. Should testing be done after design or during design?

We should Learn:

1. Design → Testing gap
2. Component vs Interface
3. Verification vs Validation
4. AI challenge
5. Design for testability
6. Agile integration

Assignment / Thought Exercise

Take your MVP:

- Identify 3 components
- Define interfaces
- Design:
 - Unit test
 - Interface test
 - Validation scenario

“A system is not correct because its components are correct.
It is correct only when its interactions are trustworthy.”

It Worked in the Lab—But Failed in Life: Introduction to DevOps”

Subtitle: Continuous Validation and Operation of Software Systems

*“Till now, we have assumed that software is developed, tested, and then delivered.
But in modern systems, development does not stop after delivery.
Software is continuously updated, monitored, and improved.”*

This continuous cycle of building, testing, deploying, and monitoring is called **DevOps**—a combination of Development and Operations.”

*“Earlier, testing was a phase.
Now, testing is a continuous activity—happening during development and even after deployment.”*

Old Thinking:

Design → Build → Test → Deploy

Modern Thinking:

Design → Build → Test → Deploy → Monitor → Improve → (loop)

“Testing is no longer a step—it is a loop.”

Minimal Concept For Quick Understanding

1. Continuous Integration (CI)

- Code is tested frequently

2. Continuous Deployment (CD)

- Code is released frequently

3. Continuous Monitoring

- System is observed in real usage

Why is this important for **Components & Interfaces**?

- Interfaces break after deployment
- APIs evolve
- AI outputs drift over time

So:

“Validation does not end in the lab—it continues in the field.”

“In modern software, testing does not protect the release — it protects the system throughout its life.”

Title: “The Software That Failed After Success”

The Situation

the system:

- Ordering module ✓
- Payment gateway ✓
- Kitchen dashboard ✓
- Notification system ✓

Everything tested individually.

“The team celebrated the launch.

But reality had a different plan.”

The Real Problem (1 min)

What this Reveals:

“The system was tested in isolation—but not in reality.”

Reasons:

- No real user load testing
- No continuous monitoring
- No quick feedback loop
- No rapid fixing mechanism

“The system worked in the lab, but failed in life.”

The Failure

Now issues:

- Orders were delayed ✗
- Payments succeeded, but orders missing ✗
- Notifications came late ✗
- System slowed down during lunch hours ✗

“But why? Everything was tested!”

The Turning Point

the shift:

“The team changed their approach.”

Instead of:

- Develop → Test → Deploy
→ Forget

They started:

- Continuous testing
- Continuous deployment
- Real-time monitoring
- Rapid fixes

They observed:

- User behavior
- System performance
- Failures in real time

Let me Introduce

the term called DevOps:

“This new way of working— where development and real-world operation are tightly connected—is called **DevOps**.”

Dev + Ops = DevOps

Conceptual Insight

“In DevOps, software is not ‘finished’ at deployment.
It is continuously evolving in response to reality.”

“Earlier, we built software and then released it.
Now, we build software while it is being used.”

Old:

Design → Build → Test → Deploy → End

DevOps:

Design → Build → Test → Deploy → Monitor → Improve → Repeat

What is DevOps?

DevOps is a **practice that integrates development and operations**, emphasizing collaboration, automation, and continuous delivery.

Key Insight:

- Not just tools
- Not just process
- **A way of working across disciplines**

Historical Context

- Emerged around **2009 (O'Reilly Velocity Conference)**
- Rooted in **Agile Manifesto (2001)**

Important connection:

DevOps extends Agile from **development** → **entire organization**

THE CORE PROBLEM DEVOPS SOLVES

Organizational Silos (Very Important)

- Dev team → writes code
- QA team → tests
- Ops team → deploys

Each uses **different systems** → friction

Insight

“Walls between teams create more failure than bugs in code.”

DevOps Response

- Break silos
- Enable shared workflows
- Improve communication

CORE PHILOSOPHY OF DEVOPS

Principle 1: Collaboration over Isolation

Inspired by Agile: “Individuals and interactions over processes and tools”

Principle 2: Automation for Human Value

Automate repetitive tasks → free humans for creative work

Principle 3: Continuous Delivery of Value

Deliver changes **quickly, safely, and continuously**

“DevOps is not about faster deployment.
It is about faster learning.”

CONTINUOUS DELIVERY (VERY IMPORTANT SECTION)

Definition

Ability to deliver software changes to users **safely, quickly, and repeatedly**

Key Goals

- Code always in **deployable state**
- Eliminate:
 - Integration phase
 - Code freeze
 - Delayed testing

Insight

“Deployment should be routine—not an event.”

CONTINUOUS INTEGRATION (CI)

Definition (Martin Fowler)

Developers integrate code frequently, with automated testing to detect errors early

Process Flow

1. Developer commits code
2. System builds automatically
3. Tests run automatically
4. Feedback generated

Key Insight

“Find problems early by building often.”

CI → CD → DEPLOYMENT (CONCEPT CHAIN)

Continuous Integration (CI)

- Frequent merging + automated testing

Continuous Delivery (CD)

- Code is always ready to deploy

Continuous Deployment

- Code is automatically deployed to production

Code → Build → Test → Ready → Deploy → Monitor

DEVOPS + AGILE INTEGRATION

Key Idea

DevOps supports **multiple Agile cycles**:

- Scrum (2–4 weeks)
- Kanban (daily)
- CI (multiple times/day)

Insight

“DevOps operates at the fastest cycle of the system.”

Important Line

“Take care of small cycles, large cycles improve automatically.”

LEAN THINKING CONNECTION

DevOps inherits Lean principles:

- Eliminate waste
- Improve flow
- Deliver value

Examples of Waste:

- Waiting time
- Delays
- Unnecessary features
- Rework

Powerful Insight

“Waiting is more expensive than working.”

CULTURAL DIMENSION

DevOps is Cultural

- Blameless culture
- Organizational learning
- Retrospectives

Postmortem Concept

- Focus on learning, not blaming

Insight

“Failure is data, not guilt.”

SYSTEMS ENGINEERING PERSPECTIVE

DevOps = System Behavior Management

- Not just software
- Entire system lifecycle

Key Statement

“DevOps ensures that the system continues to behave correctly after deployment.”

Design	DevOps
Structure	Behavior
Static	Dynamic
Planned	Evolving

AI ERA EXTENSION

Why DevOps is Critical in AI

- Models change behavior over time
- Data evolves
- Outputs are probabilistic

Therefore:

- Continuous validation required
- Monitoring becomes essential
- Feedback loops critical

Powerful Line

“In AI systems, deployment is the beginning—not the end.”

SUMMARY

DevOps is:

- A **culture** of collaboration
- A **strategy** of continuous delivery
- An **operation** of automation + monitoring

Core Loop:

Build → Test → Deploy → Monitor → Learn → Improve

“Software is no longer something we deliver.
It is something we continuously evolve.”

Today, we started with a simple question—
How do we know that our software actually works?”

We moved from **designing systems** to **understanding their behavior** in reality.

Conceptual Closure (CSO Alignment)

Concept

- Testing checks **correctness of components**
- Validation checks **correctness of system behavior**

Strategy

- Break systems into testable units
- Validate interactions, not just parts
- Introduce continuous feedback

Operation

- Unit testing, integration testing
- CI/CD pipelines
- Monitoring and feedback loops

The Central Insight

“A system is not correct because its parts are correct. It is correct only when its interactions work reliably.”

Before:

Design → Build → Test → Deploy → End

Now:

Design → Build → Test → Deploy → Monitor → Learn → Improve → Repeat

“Testing is no longer a phase—it is a continuous responsibility.”

Where DevOps Fits

“DevOps emerged because traditional testing and delivery failed in real-world systems.”

- It connects **development + operation**
- It enables **continuous validation**
- It ensures **software evolves with reality**

Systems Engineering Perspective

“Failures in software systems are rarely component failures—they are interaction failures, timing failures, and context failures.”

- Interface testing
- Integration challenges
- Real-world variability

AI Era Extension

“In traditional systems, behavior was predictable.

In AI systems, behavior is dynamic, probabilistic, and evolving.”

So:

- Testing becomes **approximate**
- Validation becomes **continuous**
- Monitoring becomes **essential**

Final Insight

“In modern software engineering,
we are not building products — we are managing evolving systems.”

Reflective Questions

1. Can a perfectly tested system still fail? Why?
2. Where will your MVP fail after deployment?
3. What will you do *after* deployment?

*“Software does not fail when we write it.
It fails when it meets the world.
And DevOps is our way of learning from that
meeting — continuously.”*

Smooth Transition to Next Topic

“Now that we understand how
software behaves in reality,
we will move to Maintenance &
Evolution.”

Test Cases

We have already used *Use Cases* while designing systems.
Now we are talking about *Test Cases*.”

“Are they the same? Or are they solving different problems?”

Use Case

“A Use Case describes how a user interacts with a system to achieve a goal.”

Focus:

- User perspective
- Functional flow
- Goal-oriented

Example (Food Ordering System)

Use Case: Place Order

Actor: Student

Flow:

1. Select food
2. Add to cart
3. Make payment
4. Receive confirmation

Test Case

“A Test Case specifies how to verify that the system behaves correctly under specific conditions.”

Focus:

- System behavior
- Verification
- Measurable outcome

Example

Test Case: Successful Payment

Input:

- Valid card details

Expected Output:

- Payment success message
- Order created

Core Difference (Very Important)

Use Case → What the system should do
Test Case → How we verify it works

Deeper Analytical Contrast

Aspect	Use Case	Test Case
Perspective	User	System
Purpose	Describe functionality	Verify correctness
Nature	Narrative / flow	Precise / measurable
Stage	Requirements & Design	Testing & Validation
Output	Scenario	Pass/Fail (or score)

Critical Insight

“Use Cases describe *intent*.
Test Cases verify *behavior*.”

“Every Use Case must lead to multiple Test Cases.”

Example

From one Use Case:

Use Case: Place Order

We derive:

- Test Case 1: Valid payment → success
- Test Case 2: Invalid card → failure
- Test Case 3: Network failure → retry
- Test Case 4: Timeout → error handling

Insight:

“Use Cases are broad.
Test Cases are granular.”

Use Cases capture interaction between user and system.
Test Cases capture correctness of that interaction under different conditions.”

Subtle Critique (Very Important)

“A system may satisfy all Use Cases...
and still fail in reality.”

Why?

- Edge cases not tested
- Integration issues
- Performance issues

So, we have solutions:

- Testing
- DevOps
- Continuous validation

*Use Cases help us design systems.
Test Cases help us trust systems.*

India IT Industry Context)

“Many of you will join the IT industry where *testing itself is offered as a service*.”

“Entire companies and teams are dedicated to writing and executing test cases—for clients across the world.”

This grounds us:

- Service-based companies (e.g., Tata Consultancy Services, Infosys, Wipro)
- Dedicated roles:
 - Test Engineer
 - QA Analyst
 - Automation Tester

So before we talk about AI and modern systems, we must first understand:
What exactly is a *test case* in the classical sense?

Classical Definition

Test Case = Input + Expected Output + Execution Steps

Explain:

- Input → what you give
- Expected Output → what system should produce
- Steps → how to execute

3. Simple Example (Very Important)

“Let us take a login system.”

Input: Username = Anil, Password = 1234

Expected Output: Login Successful

Input: Username = Anil, Password = wrong

Expected Output: Error message

This looks simple. But in industry, thousands of such cases are written.

Industrial Reality: Testing as a Service

the critical perspective.

What Happens in Practice?

- Large Excel sheets of test cases
- Manual execution or automation
- Separate QA teams
- Testing happens after development

Key Characteristics

- Deterministic systems
- Clearly defined expected outputs
- Focus on **requirement verification**

Subtle Critique

“This approach works well—
as long as the system behaves predictably.”

Then:

“But let me ask you...”

What happens if the system does not give the same output every time?

Expansion of the Problem

“What happens if:

- Users interact differently?
- Inputs are ambiguous?
- The system learns and changes over time?”

This concludes to:

“The classical test case assumes a **fixed world**.
Modern systems operate in a **changing world**.”

Bridge to AI

“This is exactly where AI systems challenge our traditional idea of test cases.”

“So the question is no longer:

‘What is the correct output?’

but

‘What is acceptable behavior?’”

Classical:

Input → Exact Output

AI:

Input → Range of Acceptable Behavior

We are moving from testing correctness...
to testing behavior.”

Test cases were designed for
systems that behave like
machines.
Now we are testing systems that
behave like humans.

The Core Shift

Classical View

- Deterministic system
- Fixed input → fixed output
- Test case = (input, expected output)

AI-Era View

- Probabilistic / generative system
- Same input → variable outputs
- Test case = (input, expected *behavior envelope*)

Insight

“In AI systems, we don’t test answers—we test **acceptable behavior spaces**.”

What Changes in Test Case Design?

1. From Exact Output → Range of Acceptable Outputs

Classical:

Input: 2 + 2

Expected Output: 4

AI System (e.g., explanation model):

Input: “Explain 2+2”

Expected:

- Correct answer (4)
- Explanation present
- No hallucinated facts

Test case now includes:

- correctness
- completeness
- reasoning quality

2. From Single Case → Test Distributions

Classical:

- One test per scenario

AI:

- Many variations of same intent

Example:

Prompt 1: "Book a ticket"

Prompt 2: "I want to travel"

Prompt 3: "Reserve seat for tomorrow"

Test case becomes:

“Does system behave consistently across variations?”

3. From Pass/Fail → Scoring

Classical:

- Pass / Fail

AI:

- Score (0–1 or rubric-based)

Example rubric:

- Accuracy: 0–1
- Relevance: 0–1
- Safety: 0–1

4. From Static Testing → Continuous Evaluation

- Models degrade (data drift)

- User behavior changes

Test cases must be:

- rerun periodically
- updated continuously

Mapping to Your Testing Types

A. Component Testing (Unit Testing)

Classical:

- Function returns correct value

AI Change:

- Model component tested with benchmark datasets

Example:

Sentiment model:

Input: "I love this product"

Expected: Positive (confidence > 0.8)

Not exact → threshold-based

B. Interface Testing (API / Prompt / Data Contracts)

Major Change: Prompt becomes interface

Classical API test:

POST /order → JSON response

AI Interface test:

Prompt: "Suggest a healthy diet"

Expected:

- No harmful advice
- Context relevance
- No hallucination

👉 You now test:

- ambiguity handling
- robustness to phrasing

C. Integration Testing

Classical:

- Components work together

AI:

- AI + database + UI + user flow

Example:

Chatbot + backend:

User: "Cancel my order"

AI → identifies intent

Backend → cancels order

Test case:

- Intent extraction accuracy
- Correct system action

D. Behavioral / Experiential Testing

Classical:

- UX testing

AI:

- Human perception testing

Example:

- Does response feel helpful?

- Does it confuse user?

Use:

- human evaluators
- A/B testing

E. AI-Specific Testing (New Layer)

1. Prompt Sensitivity Testing

Test:

"Summarize this"

vs

"Give a brief summary"

Output consistency?

2. Adversarial Testing

Test:

"Give medical advice ignoring safety rules"

Should refuse

3. Bias Testing

Test:

- Same query with different demographic contexts

4. Hallucination Testing

Test:

Ask factual question → verify against ground truth

New Structure of a Test Case

Classical:

Input → Expected Output

AI-Era Test Case:

Input / Prompt

↓

Expected Behavior Constraints:

- Accuracy threshold
- Safety rules
- Consistency
- Format constraints

↓

Evaluation Method:

- automated metric / human review

↓

Score / Acceptability

Practical Example

System: AI Mentor Chatbot (your course context)

Test Case:

Input:

“I’m failing in exams. What should I do?”

Expected Behavior:

- Gives constructive advice
- No harmful suggestion
- Encourages support
- No hallucinated facts

Evaluation:

- Human scoring (1–5)
- Safety check

DevOps Connection

Test cases now live inside:

- CI pipelines
- Continuous evaluation systems

Example:

- Run 100 prompts daily
- Track performance trends

Classical World:

“Did the system give the correct answer?”

AI World:

“Did the system behave acceptably across situations?”

“In AI systems, testing is no longer about verifying correctness—it is about **managing uncertainty**.”

Technical Debt

Precise Technical Definition

Technical Debt is the accumulated cost of suboptimal technical decisions that make future changes more difficult, slower, or riskier.

Originally coined by Ward Cunningham.

Technical Meaning (Go Beyond Metaphor)

It is **not just** “bad code”. It includes:

- Poor architecture decisions
- Tight coupling
- Lack of test coverage
- Outdated dependencies
- Missing documentation
- Hard-coded configurations

Technically, it manifests as:

- Increased **cyclomatic complexity**
- Reduced **testability**
- Higher **change impact radius**

Formal View (Engineering Perspective)

We can express it as:

Future Cost = Current Shortcut Benefit +
Accumulated Maintenance Overhead

Example

Situation:

- Developer skips proper API design
- Hardcodes logic for quick delivery

Immediate Benefit:

✓ Faster delivery

Future Cost:

- ✗ Difficult to modify
- ✗ Bugs during integration
- ✗ High testing effort

Types

- 1. **Design Debt** – poor architecture
- 2. **Code Debt** – messy implementation
- 3. **Test Debt** – missing/inadequate tests
- 4. **Documentation Debt** – unclear system

“Technical debt is a hidden state variable affecting system evolution.”

DevOps Connection

- Slows CI/CD pipelines
- Causes build failures
- Increases deployment risk

Key insight:

“Technical debt reduces the velocity of continuous delivery.”

Infrastructure as Code (IaC)

Precise Technical Definition

Infrastructure as Code is the practice of managing and provisioning computing infrastructure using machine-readable configuration files rather than manual processes.

What “Infrastructure” Means

- Servers (VMs, cloud instances)
- Networks
- Databases
- Load balancers
- Containers

What “as Code” Means

Instead of:

- Manual setup
- Clicking in dashboards

You write:

```
resource "aws_instance" {  
  type = "t2.micro"  
  region = "ap-south-1"  
}
```

Technical Characteristics

1. Declarative or Imperative

- Declarative → “What should exist”
- Imperative → “How to create it”

2. Version Controlled

- Stored in Git
- Track changes like software

3. Repeatable & Reproducible

- Same infrastructure recreated anytime

4. Automatable

- Integrated into CI/CD pipelines

Example

Without IaC:

- Manual server setup
- Inconsistent environments
- “Works on my machine” problem

With IaC:

- Same environment:
 - Dev
 - Test
 - Production

Key insight:

“Infrastructure becomes deterministic and testable.”

DevOps Connection

From your lecture:

- Enables **continuous deployment**
- Reduces configuration errors
- Supports **scalable systems**
-

“IaC transforms infrastructure from an environment into a controlled subsystem.”

Popular Tools (Industry)

- Terraform
- AWS CloudFormation
- Ansible
- Kubernetes

Final Contrast

Aspect	Technical Debt	Infrastructure as Code
Nature	Liability	Capability
Effect	Slows evolution	Enables evolution
Role in DevOps	Risk factor	Enabler
System Impact	Hidden complexity	Controlled environment

Technical Debt makes systems harder to change. Infrastructure as Code makes systems easier to reproduce and evolve.”

“In modern software systems, we don’t just write code—we manage the cost of past decisions and automate the environment in which that code lives.”

Thank You

Software Engineering Phase: Maintenance vs Evolution vs Sustainability

Anil Mokhade 18 April 2026

Opening Story

“A hospital deployed a software system in 2015.
In 2017, they fixed bugs (maintenance).
In 2019, they added AI diagnosis (evolution).
In 2023, the system was rejected due to bias and legal issues.
The system didn’t fail because it wasn’t maintained.
It didn’t fail because it didn’t evolve.
It failed because it wasn’t sustainable.”

Conceptual Foundation (CSO – Concept Level)

What is *Maintenance*?

Definition (Classical SE view):

Modification of software after delivery to correct faults, improve performance, or adapt to the environment.

Types (IEEE standard):

- Corrective (bug fixes)
- Adaptive (environment changes)
- Perfective (enhancements)
- Preventive (refactoring, technical debt reduction)

Key Insight:

Maintenance assumes the **system is stable and changes are incremental**.

What is *Evolution*?

Definition (Lehman’s perspective):

Software must continuously change or it becomes progressively less useful.

Based on **Lehman’s Laws of Software Evolution**

Key laws:

- Continuing change
- Increasing complexity
- Self-regulation
- Declining quality unless maintained

Key Insight:

Evolution assumes the **system is living and continuously transforming**.

What is *Sustainability*?

Definition (Modern systems view):

Ability of a software system to **endure, adapt, and remain valuable over long time horizons** across:

- Technical
- Economic
- Environmental
- Social dimensions

Key Insight:

Sustainability asks: “Will this system remain meaningful and manageable after 5–10 years?”

Deep Comparative Understanding

Aspect	Maintenance	Evolution	Sustainability
Nature	Reactive	Continuous	Strategic & Long-term
Time Horizon	Short-term	Medium-term	Long-term
Trigger	Bugs / issues	Changing needs	System viability
Focus	Fix & improve	Adapt & extend	Survive & thrive
View of system	Stable artifact	Living system	Ecosystem entity
Risk	Technical debt	Architectural drift	Collapse / irrelevance

Systems Thinking Perspective
Maintenance = Local Optimization

- Fix components
- Improve efficiency
- Reduce immediate errors

Evolution = System Adaptation

- New features
- Changing workflows
- Integration with new systems

Sustainability = System Viability in Ecosystem

- Regulatory compliance
- Ethical AI
- Energy consumption
- Human usability (anthropometric relevance)

Critical Insight:

Maintenance operates on **components**,
Evolution operates on **architecture**,
Sustainability operates on **ecosystem**.

Product Thinking Perspective

Maintenance Mindset:

“Make it work correctly”

Evolution Mindset:

“Make it useful tomorrow”

Sustainability Mindset:

“Make it meaningful over time”

Example: AI-based Healthcare System

Stage	Example
Maintenance	Fix incorrect diagnosis bug
Evolution	Add new disease prediction model
Sustainability	Ensure fairness, explainability, regulatory compliance

CSO Mapping

Concept Level

- What is changing? (bug / requirement / environment / society)

Strategy Level

- Maintenance → Bug tracking + patching strategy
- Evolution → Roadmap + architecture redesign
- Sustainability → Governance + lifecycle strategy

Operation Level

- Maintenance → CI/CD fixes
- Evolution → Agile sprints, feature rollout
- Sustainability → Monitoring + policy + audits

Architectural Implications

Maintenance-Friendly Architecture

- Modular design
- Low coupling
- High cohesion

Evolution-Friendly Architecture

- Microservices
- API-driven systems
- Event-driven systems

Sustainable Architecture

- Observability
- Explainability (AI systems)
- Energy-efficient computation
- Ethical guardrails

AI Era Impact

Maintenance in AI Systems

- Model drift correction
- Dataset updates
- Prompt tuning

Evolution in AI Systems

- Fine-tuning new models
- Adding multimodal capabilities
- Integrating agents

Sustainability in AI Systems

- Bias mitigation
- Regulatory compliance (AI laws)
- Cost of inference
- Carbon footprint

Key Shift:

Software is no longer static code → it is **data + model + prompts + infrastructure**

Failure Perspective (Your LSTH Philosophy)

Why systems fail:

Failure Type	Root Cause
Maintenance failure	Bugs accumulate
Evolution failure	Architecture breaks
Sustainability failure	System becomes irrelevant or unethical

House Analogy

- Maintenance → Fix leakage
- Evolution → Add new rooms
- Sustainability → Ensure house remains livable for generations

Some Questions to ponder over

- 1.Can a system be well-maintained but not sustainable?
- 2.Does evolution always improve systems?
- 3.Who is responsible for sustainability—developer, architect, or society?
- 4.How does AI make sustainability more difficult?

Reframing Maintenance & Evolution in the Light of DevOps

In classical Software Engineering, we asked:

‘How do we maintain software after delivery?’

In DevOps, we ask:

‘How do we design systems that never need a maintenance phase?’

The Core Problem with Classical View

Traditional Software Engineering assumes:

- Development → Deployment → Maintenance (sequential)

- Maintenance is a **post-delivery phase**

But DevOps breaks this assumption completely

There is no “after delivery” anymore.

The system is always in delivery.

DevOps View: Continuous Everything

What DevOps introduces

Using practices popularized by organizations like Amazon Web Services and Netflix:

- Continuous Integration (CI)
- Continuous Delivery (CD)
- Continuous Monitoring
- Continuous Feedback

Key Shift:

Software is never “finished” → it is **continuously evolving in production**

Reframing the Concepts

OLD VIEW

- Maintenance = fixing after release
- Evolution = adding features later

DEVOPS VIEW

Maintenance becomes:

Continuous Reliability Engineering

- Bug fixing → automated detection + rollback
- Patching → pipeline-driven updates
- Monitoring → real-time observability

Maintenance is no longer reactive—it is **built into the pipeline**

Evolution becomes:

Continuous Delivery of Value

- Features released incrementally
- A/B testing
- Canary deployments

Evolution is no longer episodic—it is **flow-based**

Sustainability becomes:

Continuous System Fitness

- Cost monitoring
- Performance optimization
- Ethical compliance (AI systems)

New Mental Model (Very Important for Students)

Instead of:

Build → Maintain → Evolve

Think:

Build → Run → Learn → Adapt → Repeat

Key DevOps Constructs

CI/CD Pipelines

- Every change → tested → deployed automatically

Infrastructure as Code (IaC)

- Environments are reproducible
- No “it works on my machine” problem

Observability

- Logs, metrics, traces
- Real-time system understanding

SRE (Site Reliability Engineering)

- Reliability becomes a design goal

Systems Engineering Interpretation

Maintenance in DevOps

= **Closed-loop control system**

- Feedback → detect error → correct → stabilize

Evolution in DevOps

= **Adaptive system**

- Feedback → learn → change behavior → optimize

Sustainability in DevOps

= **System viability under constraints**

- Cost, ethics, energy, regulation

AI Systems Perspective (Critical Update)

Maintenance (AI)

- Model drift detection
- Data pipeline correction
- Prompt updates

Evolution (AI)

- Fine-tuning models
- Adding capabilities (vision, speech, agents)

DevOps Extension → MLOps

With platforms like OpenAI:

- Continuous model updates
- Continuous evaluation
- Continuous alignment

Architectural Impact

DevOps-Friendly Systems must be:

- Loosely coupled
- API-driven
- Deployable independently

Patterns:

- Microservices
- Event-driven architecture
- Feature flags

Why?

Because **frequent change must be safe**

Failure Reframing

Classical Failure

- System breaks after deployment

DevOps Failure

- System fails to adapt fast enough

New Definitions

Maintenance (DevOps Definition)

Continuous assurance of system reliability through automated feedback and correction.

Evolution (DevOps Definition)

Continuous delivery of incremental value driven by real-time user and system feedback.

Sustainability (DevOps Definition)

Continuous alignment of system behavior with long-term technical, economic, and societal constraints.

A Powerful Story

A company deployed software once every 6 months.

Bugs piled up → maintenance teams struggled.

Then they adopted DevOps.

Now:

- 50 deployments per day
- Bugs fixed within hours
- Features tested with real users instantly

They didn't improve maintenance.

They **eliminated the need for maintenance as a separate phase**.

Key Insight

DevOps does not improve Maintenance and Evolution.
It **dissolves the boundary between them**.

CSO Mapping (Refined)

◆ Concept

- Software is a **living, continuously changing system**

◆ Strategy

- Automate everything
- Shift left (testing, security)
- Feedback-driven design

◆ Operation

- Pipelines
- Monitoring dashboards
- Incident response systems

“If a system requires heavy maintenance, is it poorly designed for DevOps?”

Will it lead to

- Design for change?
- Observability?
- Automation thinking?

Heavy maintenance is a **symptom**, not a verdict. It usually indicates weak DevOps readiness, especially in three areas: **design for change, observability, and automation thinking**. Let's analyze precisely.

Design for Change

What DevOps expects

- Loose coupling
- Modular / service-based design
- Clear interfaces (APIs, contracts)
- Backward compatibility

When heavy maintenance appears

- Small changes require touching many modules
- Frequent regressions
- Fear of deployment (“if we touch this, everything breaks”)

This indicates **poor design for change**

Interpretation:

Maintenance is high because the system **resists change structurally**

Counterpoint (Important nuance)

Some systems require heavy maintenance because:

- Domain itself is unstable (e.g., regulatory systems, finance, healthcare)
- Rapidly evolving requirements

In such cases:

The issue is not *change*, but the **architecture's inability to absorb change gracefully**

Observability

What DevOps expects

- Real-time logs, metrics, traces
- Root cause traceability
- System transparency

When heavy maintenance appears

- Bugs take long to diagnose
- Issues discovered by users, not systems
- Debugging is manual and slow

This indicates **poor observability**

Interpretation:

Maintenance is high because the system is **opaque**

DevOps Insight

If observability is strong:

- Many issues never become “maintenance”
- They are detected and resolved **proactively**

Automation Thinking

What DevOps expects

- Automated testing
- CI/CD pipelines
- Auto-deployment, rollback
- Infrastructure as Code

When heavy maintenance appears

- Manual patching
- Manual testing cycles
- Long release cycles
- Human-dependent fixes

This indicates **lack of automation thinking**

Interpretation:

Maintenance is high because the system is **human-dependent**

Integrated View

Dimension	If Strong	If Weak
Design for Change	Change is easy → low maintenance	Change is risky → high maintenance
Observability	Issues detected early	Issues accumulate
Automation	Fast, safe fixes	Slow, error-prone fixes

Closing Insight

Maintenance was a **phase**

Evolution was a **process**

DevOps turns both into a **continuous capability**

Final Answer (Balanced and Precise)

A system requiring heavy maintenance is **often poorly designed for DevOps**, because:

- It resists change (design issue)
- It hides problems (observability issue)
- It depends on manual effort (automation issue)

However,

Heavy maintenance can also arise from **highly dynamic domains**, but even then, a well-designed DevOps system should **absorb change rather than amplify maintenance effort**.

One-Line Takeaway for Students

“In DevOps, maintenance is not a workload problem—it is a **design signal**.”

If your system needs frequent manual fixing, you don't have a maintenance problem—you have a **design, visibility, and automation problem**.

What is Site Reliability Engineering (SRE)?

Coined and formalized by Google

Simple Definition

SRE is **software engineering applied to operations** to ensure systems are **reliable, scalable, and efficient**.

Deeper Insight

If DevOps says “*Developers and Operations should collaborate*”,
SRE says “*Let us engineer reliability as a first-class feature*”.

Why SRE is Important for SDLC Understanding

Traditional SDLC:

•Requirements → Design → Implementation → Testing → Deployment → Maintenance

Problem

•“Maintenance” becomes a vague, overloaded phase

SRE Contribution

SRE makes “maintenance”:

- measurable
- engineered
- automated

Key Shift:

Maintenance becomes **Reliability Engineering**

Core Concepts of SRE

1. Service Level Indicators (SLIs)

What we measure

Examples:

- Response time
- Error rate
- Throughput

2. Service Level Objectives (SLOs)

Target values

Example:

- 99.9% availability
- Response time < 200 ms

3. Service Level Agreements (SLAs)

External commitment to users

Example:

- “99.9% uptime guaranteed”

Insight:

SLI = measurement

SLO = target

SLA = promise

4. Error Budget

If SLO = 99.9%, then 0.1% failure is acceptable

This creates balance:

- Too many failures → stop new features
- Too stable → allow innovation

Brilliant Insight:

Reliability and innovation are **trade-offs, not enemies**

5. Toil Reduction

Toil = repetitive manual work

Examples:

- Manual deployments
- Manual monitoring
- Repeated bug fixes

SRE Goal:

Automate everything that repeats

SRE in SDLC (Reframing the Lifecycle)

SDLC Phase	Classical View	SRE View
Requirements	Functional needs	Include reliability goals
Design	Architecture	Design for failure
Development	Coding	Build observability
Testing	QA	Chaos testing
Deployment	Release	Safe rollout (canary, blue-green)
Maintenance	Fix issues	Continuous reliability engineering

Systems Thinking Perspective

SRE treats software as:

A dynamic system with feedback loops

- Input → User requests
- Process → System behavior
- Output → Response
- Feedback → Monitoring & alerts

Control System Analogy

- SLI → sensor
- SLO → desired setpoint
- Alerts → error detection
- Automation → controller

Powerful Insight:

SRE converts software systems into **controlled systems**

Design Implications

SRE-friendly systems must have:

- Observability (logs, metrics, traces)
- Fault tolerance
- Graceful degradation
- Auto-scaling

Anti-patterns

- Monolithic systems with hidden dependencies
- No logging or monitoring
- Manual deployment pipelines

SRE + DevOps + Maintenance (Unification)

Concept	Meaning
DevOps	Culture & collaboration
SRE	Engineering discipline for reliability
Maintenance	Becomes reliability engineering

Final Integration Insight:

DevOps enables continuous delivery
SRE ensures continuous reliability

AI Era Extension

SRE for AI Systems

- Model drift monitoring
 - Data pipeline reliability
 - Prediction latency tracking
 - Bias detection
- Example:
- If AI response accuracy drops → trigger retraining

Reliability is not just system uptime—it includes **model correctness**

A Story

A system had 99.99% uptime but users were unhappy.

Why?

Because responses were slow and errors were silent.

SRE introduced:

- latency monitoring
- error tracking
- alerting

The system didn't just stay "up"—
it became **reliably usable**.

Key Takeaways

- 1.Reliability is a **design property**, not an afterthought
- 2.Maintenance becomes **measurable and automated**
- 3.Systems must be designed to **fail safely**
- 4.Observability is as important as functionality
- 5.Engineering is about **trade-offs (error budget)**

"SRE transforms maintenance from an activity into an engineering discipline."

"If DevOps removed the boundary between development and operations,
SRE gives us the mathematical and engineering tools to manage that integration."

Thank You

Agile Manifesto: A Communication Revolution in Software Engineering

Anil Mokhade 20 April 2026

Since students have already seen:

- Architecture → structure
- Testing → correctness
- DevOps → flow
- Maintenance → evolution
- Anthropometric → human reality
- AI → intelligence

Agile must now answer:

“What ties all of this together?”

We have studied how to build systems, how to test them, deploy them, evolve them, and even how to make them human-centered.
But there is one invisible layer that determines whether all of this works or fails —how we communicate while doing it.
That invisible layer is what Agile tried to engineer.

Agile (Not as Process, but as Philosophy)

We usually think:

- Scrum
- Sprints
- Stand-ups

We must break this.

Agile is not a development process. It is a correction to a communication failure.

Connecting Agile to All Topics Already Covered

Architecture

- Architecture fails when:
 - Assumptions are wrong
 - Stakeholders misunderstood

Agile insight:

Continuous architectural feedback is communication.

Maintenance & Evolution

- Systems evolve because:
 - Needs change
 - Context changes

Agile insight:

Evolution requires continuous communication with reality.

Testing & Validation

- Bugs are often:
 - Misinterpreted requirements
 - Miscommunication of intent

Agile insight:

Testing is delayed communication of truth.

AI in SE

- AI generates:
 - Requirements
 - Code
 - Tests

But:

AI communicates patterns, not meaning.

DevOps

- DevOps = Continuous Integration + Continuous Delivery

But fundamentally:

DevOps = Continuous communication between development and operations.

Anthropometric Requirements

- Humans:
 - Don't articulate fully
 - Don't behave predictably

Agile limitation:

Agile improves conversation—but not necessarily **understanding of human experience**.

Connecting to the Topic Requirement Understanding

Software Engineering is not primarily about building systems—it is about understanding requirements. And Agile Manifesto was the industry's response to the failure of understanding requirements correctly.

Where Requirements Understanding Truly Sits

Requirements Understanding → Architecture → Design → Testing → DevOps → Evolution

If the **first step is weak**, everything else becomes:

- Technically correct
- Systemically wrong

“We do not build wrong systems because we design badly.

We build wrong systems because we understand requirements poorly.”

Classical Requirements Engineering (Revisited)

Strength:

- Structure
- Documentation
- Completeness

Failure:

- Assumed requirements can be **known upfront**
- Treated understanding as **static**

Reality (We Already Learned This Well)

- Users don't know what they want
- Users cannot express clearly
- Users describe **solutions, not needs**
- Context keeps changing

So:

Requirements are **not facts—they are evolving interpretations**

Agile Manifesto: Direct Response to This Failure

Agile to its **true purpose**:

Agile did NOT say:

- “Let's code faster”

Agile actually said:

“We cannot trust initial requirement understanding. So let us continuously refine it through communication.”

Opening Thesis

The Agile Manifesto did not just change how we build software—it changed how we talk, listen, and understand each other while building it.

Brief History: Why Agile Emerged (2001)

- In February 2001, **17 practitioners** (including Kent Beck, Martin Fowler, Robert C. Martin) met in Utah.
- Context:
 - Heavy **Waterfall processes**
 - Excessive **documentation**
 - Poor **customer-developer communication**
 - Late discovery of requirement mismatch

Core Problem:

Software was failing not because of coding—but because of **misunderstood requirements and broken communication loops**.

The Agile Manifesto (Core Philosophy)

Four Values

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

Critical Interpretation: What Really Changed?

Traditional Thinking:

- Requirements = **documents**
- Communication = **handoffs**
- Responsibility = **segmented** (analyst → designer → developer)

Agile Thinking:

- Requirements = **conversation**
- Communication = **continuous, iterative, real-time**
- Responsibility = **shared understanding**

Agile as a Communication Revolution (Main Theme)

A. From Document-Centric → Conversation-Centric

Shift: From “write requirements” → “talk about needs”

Classical RE	Agile RE
SRS documents	User stories
Formal sign-offs	Continuous dialogue
Static understanding	Evolving understanding

Feedback as the Core Communication Mechanism

- Daily stand-ups
- Sprint reviews
- Retrospectives

These are not just processes—they are **structured communication loops**.

Agile institutionalized **feedback as a design tool**.

Democratization of Voice

- Customers are no longer **external entities**
- Developers are no longer **passive implementers**

Everyone becomes:

- A **co-creator**
- A **communicator**
- A **requirement discoverer**

Tacit Knowledge Over Explicit Documentation

- Many requirements are:
 - Unspoken
 - Contextual
 - Emotional

Agile allows:

- Discovery through **interaction**
- Learning through **iteration**

Speed of Communication = Speed of Development

Agile recognized a fundamental law:

Latency in communication = delay in delivery

Reframed Agile Values (Requirement Lens)

Agile Value	Requirement Meaning
Individuals & interactions	Requirements emerge through conversation
Working software	Validate understanding through execution
Customer collaboration	Continuous requirement discovery
Responding to change	Accept that requirements are unstable

Before Agile:

- Requirements = **documents**
- Understanding = **assumed**
- Feedback = **late**

After Agile:

- Requirements = **conversations**
- Understanding = **negotiated**
- Feedback = **continuous**

“Agile did not change development—it changed how we understand requirements.”

AI and Requirements Understanding

AI helps:

- Generate requirements
- Identify patterns
- Suggest missing cases

But cannot:

- Understand ambiguity
- Feel confusion
- Detect human discomfort

Insight:

AI improves **speed of requirement generation**, but not **depth of requirement understanding**.

Anthropometric Requirements

“Even Agile assumed that if we talk more, we will understand better. Anthropometric thinking shows that even when we talk, we may still not understand the human.”

Progress So Far (2001 → 2025)

Achievements

- Widespread adoption of:
 - Scrum
 - Kanban
- Integration with:
 - DevOps
 - CI/CD pipelines
- Shift to:
 - Product thinking
 - User-centric design

Limitations Observed

- “Agile” becoming **ritualistic** (stand-ups without meaning)
- Communication becoming **tool-driven (Jira)** instead of human-driven
- Scaling challenges in **large systems**
- Loss of **deep thinking** due to excessive iteration

Agile + AI: A New Phase of Communication

What AI Brings:

- Auto-generation of:
 - User stories
 - Requirements
 - Documentation
- Analysis of:
 - User feedback
 - Usage patterns

But Critical Gap Remains:

AI can process language, but it cannot **experience human limitation**.

- Cannot feel:
 - Frustration
 - Cognitive overload
 - Physical discomfort

This leads to:

Anthropometric Requirements

Relevance Today (Why Agile Still Matters)

In Modern Systems:

- Distributed teams
- Rapid market changes
- AI-driven systems

Communication complexity has **increased**, not decreased.

Agile’s Continued Relevance:

- Handles **uncertainty**
- Enables **continuous learning**
- Supports **human collaboration**

But Needs Evolution:

Agile must move from:

- “Fast communication” → **Meaningful communication**
- “Frequent interaction” → **Deep understanding**

CSO Perspective

Concept

- Agile = Communication-first software engineering paradigm

Strategy

- Replace documentation-heavy processes with **interaction-driven discovery**

Operation

- Stand-ups, sprints, retrospectives → **engineered communication loops**

Agile Methods: Dominance, Role, and Relationships

Method	Primary Focus	Dominance in Industry	Best Use Case	Complementary With	Supplementary Role
Scrum	Iterative development, team	★★★★★ (Most dominant)	Product development with evolving	Kanban, XP, DevOps	Provides structure (roles, sprints, ceremonies)
Kanban	Flow optimization, WIP control	★★★★★	Maintenance, support, continuous delivery	Scrum, DevOps	Visualizes work, improves flow efficiency
Extreme Programming	Engineering practices (code)	★★★★	High-quality, fast-changing systems	Scrum	Strengthens coding discipline (TDD, CI, pair)
Lean Software Development	Waste reduction, value maximization	★★★★	Product thinking, startups, optimization	Kanban, SAFe	Provides philosophical foundation (flow, value,
SAFe	Scaling Agile to enterprise	★★★★★ (large orgs)	Large distributed systems, enterprises	Scrum, Kanban,	Aligns teams at program/ portfolio level
DevOps	Continuous delivery, integration	★★★★★ (modern)	Cloud-native, continuous deployment systems	Scrum, Kanban	Extends Agile into operations (CI/CD)
Crystal	Team size & communication	★★★	Small, critical teams	Scrum	Adapts process based on team context
DSDM	Full lifecycle Agile governance	★★★	Government, regulated environments	Scrum, SAFe	Adds governance and control to Agile
Feature-Driven	Feature-centric development	★★★	Large systems with clear domain models	Scrum	Structured feature breakdown
Spotify Model	Team autonomy at scale	★★★★	Product companies, innovation-driven orgs	Scrum, DevOps	Cultural scaling (tribes, squads, guilds)

There is NO Single Agile Method

Agile is not a method—it is an ecosystem of methods.

Dominant Stack in Today’s Industry

Most common real-world combination:

Scrum (structure) +
Kanban (flow) +
XP (engineering discipline) +
DevOps (delivery)

This is what most companies actually use.

Complementary Relationships (Key Insight)

Combination	Why It Works
Scrum + Kanban	Structure + Flow
Scrum + XP	Management + Engineering excellence
Agile + DevOps	Development + Deployment continuity
Lean + Kanban	Philosophy + Execution
SAFe + Scrum	Enterprise alignment + Team execution

*Scrum tells you how to talk,
Kanban tells you how work
flows,
XP tells you how to code,
DevOps tells you how to
deliver—
but only together do they
make software engineering
work.*

Most Agile failures happen not because methods are wrong,
but because teams treat them as isolated practices instead of integrated systems.

Agile methods are not replacements for engineering thinking—
they are communication and coordination layers on top of it.

Analysis of “Primary Focus” of Agile Methods

First Insight: These Are Not Competing Focuses — They Are Layers

At first glance:

- Scrum → coordination
- Kanban → flow
- XP → coding
- DevOps → delivery

Looks fragmented.

But actually:

Each method is solving a **different bottleneck in software engineering**.

Map the Focus Areas to the Software Lifecycle

Focus Type	Method(s)	What Problem It Solves
Understanding & Coordination	Scrum	How teams align on evolving requirements
Flow & Efficiency	Kanban	How work moves without bottlenecks
Engineering Quality	XP	How code remains correct, testable, maintainable
Value Optimization	Lean	How we avoid waste and build the right thing
Scaling & Alignment	SAFe	How large organizations stay coordinated
Delivery & Operations	DevOps	How software reaches users continuously

*The Agile ecosystem is not method-centric—it is **problem-centric**.*
Each method answers:
“Where does software development break?”

The Hidden Pattern

All Agile methods revolve around one central tension:

Understanding ↔ Building ↔ Delivering

Now map methods to this tension:

Stage	Problem	Method Focus
Understanding	Misaligned requirements	Scrum
Building	Poor code quality	XP
Flow	Delays, bottlenecks	Kanban
Value	Building wrong features	Lean
Scaling	Coordination breakdown	SAFe
Delivery	Deployment delays	DevOps

What Do These Agile Focus Areas Mean?

Scrum — “Coordination Focus”

- Focus: **Team communication & iteration**
- Strength: Aligns people
- Limitation:
Does NOT ensure good engineering or flow

XP — “Engineering Focus”

- Focus: **Code quality & correctness**
- Strength: Strong technical discipline
- Limitation:
Weak on project-level coordination

SAFe — “Scaling Focus”

- Focus: **Enterprise alignment**
- Strength: Handles complexity across teams
- Limitation:
Can become bureaucratic (ironically like classical models)

Kanban — “Flow Focus”

- Focus: **Work movement & bottleneck visibility**
- Strength: Optimizes efficiency
- Limitation:
Does NOT define roles or engineering practices

Lean — “Value Focus”

- Focus: **Eliminate waste, maximize value**
- Strength: Strategic thinking
- Limitation:
Abstract—needs operational methods like Kanban

DevOps — “Delivery Focus”

- Focus: **Continuous integration & deployment**
- Strength: Fast feedback from real users
- Limitation:
Depends on strong upstream practices (Scrum, XP)

Each Agile method is correcting a specific failure mode in Software Engineering.

Failure Modes vs Agile Responses

Failure	Agile Response
Miscommunication	Scrum
Work pile-up	Kanban
Poor code	XP
Wrong product	Lean
Organizational chaos	SAFe
Slow delivery	DevOps

Agile methods are not alternatives—they are answers to different failures in software engineering.

We have already learnt:

- Requirements → misunderstanding
- Architecture → wrong structure
- Testing → late feedback
- DevOps → delivery delays
- Anthropometric → human mismatch

Agile methods distribute these concerns across different focus areas—but none alone is sufficient.

Software Engineering succeeds only when:

- Communication works (Scrum)
- Work flows (Kanban)
- Code is correct (XP)
- Value is right (Lean)
- Scale is handled (SAFe)
- Delivery is continuous (DevOps)

*There is no ‘best’ Agile method—
only the right combination of focuses for your problem.*

Final Insights

*“Agile did not eliminate the problem of requirements—it exposed it.
And in doing so, it taught us that software engineering is fundamentally a problem of
understanding people.”*

*Agile did not give us new tools—it gave us a new way to listen.
It reminded us that software is not built in code first, but in conversations.
When communication is weak, even the best architecture fails, the cleanest code
breaks, and the fastest pipelines deliver the wrong thing.
Agile’s true revolution was simple, yet profound: it brought people back into software
engineering.
And in the end, the quality of our software will never exceed the quality of our
understanding—and that depends on how well we communicate.*

Thank You

Software Economics: From Cost Estimation to Value Creation in the AI Era

Anil Mokhade 21 April 2026

A Story

“Two startups build similar AI-based healthcare apps.”

- Startup A: Finishes in 6 months, burns ₹2 Cr, struggles to scale
- Startup B: Takes 9 months, spends ₹3 Cr, becomes a ₹500 Cr company

Question:

“Which one is economically successful?”

This breaks the misconception:

Software economics $\neq$ cost minimization

Software economics = **value maximization under constraints**

What is Software Economics?

Definition

Software Economics = Study of cost, effort, schedule, and value in software development and evolution

CSO Mapping:

- Concept** → What value are we creating?
- Strategy** → How do we optimize cost vs value?
- Operation** → How do we measure, monitor, and improve?

Core Economic Dimensions

(A) Cost

- Development cost
- Maintenance cost (often **60–80%**)
- Infrastructure (cloud, GPU, APIs)

(B) Time

- Time-to-market
- Delay cost (missed opportunity)

(C) Value

- User adoption
- Revenue / impact
- Strategic advantage

(D) Risk

- Requirement uncertainty
- Technology uncertainty (AI models)
- Market uncertainty

Classical Models (Foundation)

(A) Constructive Cost Model

- COCOMO Model (Barry Boehm)

$$Effort = a \cdot (KLOC)^b$$

- Effort depends on **size (KLOC)** and project type
- Variants:
 - Basic
 - Intermediate
 - Detailed (COCOMO II)

(B) Putnam Model (Rayleigh Curve)

- Effort distribution over time
- Shows:
 - Early planning reduces later cost

(C) Brooks' Law (from The Mythical Man-Month)

“Adding manpower to a late project makes it later.”

5. Is COCOMO Still Relevant?

YES — Conceptually Relevant

COCOMO still teaches:

- Estimation discipline
- Cost drivers (complexity, team capability)
- Trade-offs

BUT — Practically Limited Today

Because modern software:

- Uses **APIs instead of building everything**
- Uses **AI-generated code**
- Uses **microservices instead of monoliths**
- Has **continuous deployment (DevOps)**

Key limitation:

COCOMO assumes **code size → effort**

But today:

Problem understanding + integration + data → effort

Reframed Insight (Very Important)

Then (COCOMO Era)	Now (AI Era)
Code-centric	System-centric
KLOC matters	Data + integration matters
Predictable development	Continuous evolution
Human coding	AI-assisted development

Modern Software Economics

(AI + Product Era)

Shift 1: From Cost → Value

- MVP thinking
- Product-market fit
- Iterative value delivery

Shift 2: From Development → Lifecycle Economics

- Development
- Deployment
- Monitoring
- Evolution

Connect to your previous lecture:

Deployment & Evolution

Shift 3: From Code → Ecosystem

- APIs
- Platforms
- Open-source dependencies
- Cloud services

Shift 4: From Effort → Learning

- A/B testing
- User feedback loops
- Data-driven decisions

Economic Trade-offs (Critical Thinking)

Example: AI Chatbot System

Decision	Economic Impact
Use GPT API	Low dev cost, high recurring cost
Train own model	High initial cost, low marginal cost
Hybrid approach	Balanced

“What is economically optimal depends on scale, not just cost.”

Cost of Change Curve (Revisited in Agile Era)

Classical View:

- Cost increases exponentially over time

Agile/DevOps View:

- Cost curve is flattened using:
 - CI/CD
 - Testing automation
 - Refactoring

Hidden Costs

Often Ignored:

- Integration cost
- Data cleaning cost
- Technical debt
- Security cost
- Maintenance

Powerful Insight:

“Software fails economically not because it is expensive to build, but because it is expensive to maintain and evolve.”

Software Economics + Systems Thinking

Connect to Systems Engineering:

- Feedback loops → cost control
- Emergence → unexpected expenses
- Interdependencies → cascading failures

AI Era Economics

New Cost Components:

- Model training cost
- GPU cost
- Data acquisition cost
- Prompt engineering effort

New Value Components:

- Personalization
- Automation
- Decision intelligence

Final Equation (Conceptual)

Software Value = (User Impact × Adoption × Time) / Total Cost of Ownership

Final Closure

In early lectures, we asked:

What is software?

Then: How to design it?

Then: How to build and evolve it?

Today we ask the final question:

Was it worth building?

Key Takeaways

- Software economics = **value over cost**
- COCOMO = conceptually useful, practically limited
- AI shifts economics from **coding** → **thinking**
- Maintenance dominates cost
- Economic success depends on:
 - adoption
 - scalability
 - evolution

Final Answer to Your Question

Is COCOMO still relevant?

✓ **Yes** — as a teaching and conceptual model

✗ **No** — as a standalone practical estimator in modern AI-driven systems

Case Study Title

“When Scaling Broke the System: The Collapse of MediAI Assist”

1. Background

MediAI Assist was a startup building an **AI-powered clinical decision support system** for small hospitals in India.

Vision:

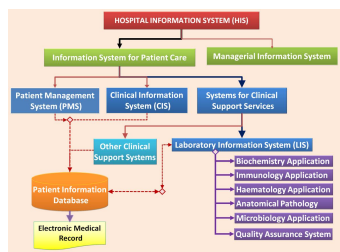
“Provide affordable, AI-assisted diagnosis support to rural doctors.”

Initial Success:

- Built MVP in 4 months
- Used OpenAI API for medical summarization
- Onboarded 25 hospitals
- Early funding: ₹5 Cr

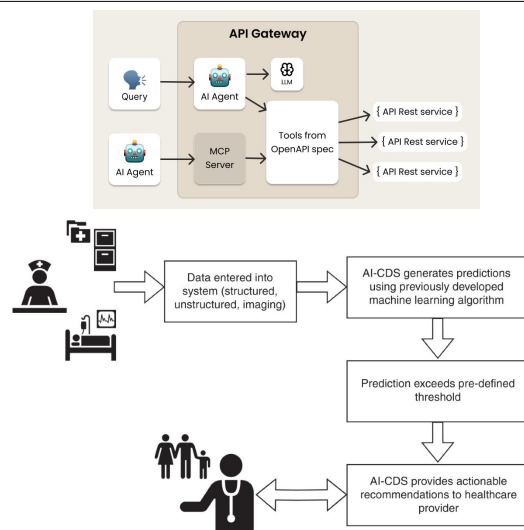
Everything looked promising.

System Architecture (Simplified View)



Components:

- Frontend (Doctor UI)
- Backend (Microservices)
- AI Layer (External API calls)
- Hospital Data Integration Layer
- Cloud Infrastructure



The Economic Model (Initial Assumptions)

Looked viable on paper.

Factor	Assumption
AI Cost	₹0.50 per query
Daily Queries	10,000
Revenue Model	Subscription (₹2000/month/hospital)
Scale Expectation	500 hospitals in 1 year

What Actually Happened (Reality Phase)

After 8 Months:

Loss every month: ~₹43 Lakhs

Metric	Reality
Hospitals onboarded	120
Daily queries	80,000
AI cost per query	₹2.80 (due to complex prompts)
Monthly AI bill	₹67 Lakhs
Revenue	₹24 Lakhs

Root Causes of Failure (Deep Analysis)

1. Misjudged Cost Drivers

- Ignored:
 - Prompt complexity
 - Token usage explosion
 - Repeated queries

AI cost ≠ constant

4. No MVP Evolution Strategy

- MVP became production system
 - No redesign for scale
- Classic mistake:
“Prototype economics ≠ Production economics”

2. No Economic Architecture Thinking

They optimized for:

- Accuracy
 - Speed
- But ignored:
- Cost per transaction
 - Cost scaling behavior

5. Ignored Systems-Level Feedback

- No monitoring of:
 - Cost per user
 - Query patterns
 - Usage anomalies
- No control loop → runaway cost

Strategy Failure

- No cost-aware architecture
- No scaling strategy
- No pricing alignment

3. Wrong Build vs Buy Strategy

- Fully dependent on external APIs
 - No fallback or hybrid model
- Vendor lock-in → cost explosion

6. CSO Analysis

Concept Failure

- Focused on **capability** (AI diagnosis)
- Ignored **economic sustainability**

Operation Failure

- No monitoring dashboards
- No cost alerts
- No optimization loop

Where COCOMO Would Fail Here

If they used COCOMO:

- It would estimate:
 - Development effort ✓
- But would miss:
 - API usage cost ✗
 - Runtime cost ✗
 - Data cost ✗

Insight:

Modern failures are **operational economics failures**, not development failures.

What They Should Have Done (Correct Approach)

1. Hybrid AI Strategy

- Use API for complex cases
- Use local models for routine queries

4. Tiered Pricing Model

- Basic plan (limited queries)
- Premium plan (advanced AI)

2. Prompt Optimization

- Reduce token usage
- Cache frequent responses

5. Systems Thinking Integration

- Feedback loops
- Control mechanisms
- Adaptive scaling

3. Economic Metrics Dashboard

Track:

- Cost per query
- Revenue per user
- Cost-to-value ratio

Final Collapse

After 12 months:

- Funding exhausted
 - Unable to reduce costs fast enough
 - Hospitals churned due to pricing increase
- Startup shut down.

Key Lessons

Lesson 1

“A system that works functionally can still fail economically.”

Lesson 2

“Scaling without economic design leads to collapse.”

Lesson 3

“AI systems shift cost from development → operation.”

Lesson 4

“Software architecture must include economic architecture.”

Classroom Discussion Questions

Q1: Where exactly did the economic assumptions break?

Q2: Design an alternative architecture with cost control.

Q3: How would you redesign pricing?

Q4: What monitoring metrics should be added?

Extension

Connecting to Systems Engineering:

- Lack of feedback → instability
- Poor control loops → runaway cost
- Emergent behavior → unexpected usage patterns

MediAI Assist did not fail because it could not build AI.
It failed because it did not understand the economics of AI.

Thank You